

Machine Learning-Based Preconditioners for Dirac Operator Inversion in Lattice QCD

Ole Wolsing Kaa (olkaa21@student.sdu.dk)

Master Thesis
IMADA

Supervisor: Benjamin Jäger



Abstract

Contents

Contents	1
1 Introduction	2
2 Iterative methods for linear systems	2
2.1 Conjugate Gradient	2
2.1.1 Convergens	5
2.1.2 Preconditioning	7
2.2 CG Variants	8
2.2.1 Normal Equation	8
2.2.2 Bi-Conjugate Gradient	8
2.2.3 Conjugate Gradient Squared	9
2.2.4 Bi-Conjugate Gradient Stabilized	10
2.2.5 Comparison	11
3 Exploration of learning preconditioners	11
3.1 Structure of the preconditioner	11
3.2 Initial learning algorithm	14
3.2.1 Better exploration of the preconditioner	16
3.3 Discrete laplace operator	17
3.4 Improvement functions	21
3.4.1 A better comparison	25
3.5 New type (Jacobi shift)***	26
3.5.1 Plateau in learning	29
3.6 Allowing disimprovement	30
3.7 Number of partitions	32
3.7.1 Fewer	34
3.8 Number of diagonals	35
3.9 Super- and subdiagonals	36
3.9.1 Number of partitions	38
3.9.2 Symmetry	40
3.10 Testing the algorithm on larger systems	41
4 Conclusion	43
5 Appendix	43
6 Bibliography	43

1 Introduction

2 Iterative methods for linear systems

Before we will try to learn a preconditioner, we first need some background theory on the iterative solvers for linear system. Both to explain what preconditioning a linear system mean and why it can accelerate the convergens of such methods, and the differences and the condition the solvers have to best chose a suitable method that has the best possible chance of getting good results for both the linear systems themselves and in trying to learn a preconditioner for the systems.

2.1 Conjugate Gradient

The Conjugate Gradients method, often shortened to CG, is iterative method for solving linear systems of the form $Ax = b$, where A is an $n \times n$ invertible positive definite symmetric matrix, x and b being vectors of size n , without calculating A^{-1} . To explain how CG work we need to start with another iterative solver Richardson iterations, which is a uses a simple recursive equation to find an approximation for for the value of x

$$x_{i+1} = x_i + r_i, \quad (1)$$

with r_i being the residual defined as $r_i = b - Ax_i$ and a initial x_0 . This method work on the condition that $\|I - A\| < 1$, [8], which is for a matrix $A \in \mathbb{R}^{n \times n}$ the matrix norm defined as

$$\|A\| = \sup_{y \in \mathbb{R}^n, y \neq 0} \frac{\|Ay\|}{\|y\|}$$

With a given x_0 we can see that the recursive equation,(1), is the sum of all previous residuals and x_0 , [8]

$$\begin{aligned} x_{i+1} &= x_{i-1} + r_{i-1} + r_i = x_0 + r_0 + r_1 + \cdots + r_i \\ &= x_0 + \sum_{j=0}^i (I - A)^j r_0 \end{aligned}$$

is in the subspace span by $x_0 + \text{span}\{r_0, Ar_0, \dots, A^i r_0\}$, where the right part is called the i th dimensional Krylov subspace generated by A and r_0 , denoted as $\mathcal{K}^i(A, r_0)$. Which means that Richardson iterations approximations come from Krylov subspaces of increasing dimension plus x_0 . We can now almost explain CG as it a Krylov subspace method with Ritz-Galerkin condition for the residuals, which is that x_i is constructed such that the corresponding residual is orthogonal to the current Krylov subspace, [8].

$$b - Ax_i \perp \mathcal{K}^i(A; r_0). \quad (2)$$

To do this one would need to create an orthonormal basis for the Krylov subspace with r_0 as the initial vector, this can be done with Arnoldi algorithm, [7], Algorithm 1. This algorithm is a modified version of the standard Gram-Schmidt procedure for creating orthonormal basis for a vector space.

Algorithm 1 Arnoldi algorithm [8].

```

 $v_1 = \frac{r_0}{\|r_0\|_2}$ 
loop  $j = 1, \dots, m - 1$ 
     $t = Av_j$ 
    loop  $i = 1, \dots, j$ 
         $h_{i,j} = v_i^T t$ 
         $t = t - h_{i,j}v_i$ 
    end loop
     $h_{j+1,j} = \|t\|_2$ 
     $v_{j+1} = \frac{t}{h_{j+1,j}}$ 
end loop

```

With this algorithm we get a relation between the linear systems matrix A , a matrix V_m with columns made from the vectors v_1 to v_m and an $m \times m - 1$ upper Hessenberg matrix $H_{m,m-1}$, which is a matrix with all zero entries below the first subdiagonal, with $h_{i,j}$ as entries, $H_{m-1,m-1}$ denotes $H_{m,m-1}$ without the last row,

$$AV_{m-1} = V_m H_{m,m-1} \quad (3)$$

$$= V_{m-1} H_{m-1,m-1} + h_{m,m-1} v_m e_m^T \quad (4)$$

$$V_{m-1}^T AV_{m-1} = H_{m-1,m-1} \quad (5)$$

Note that if A is symmetric then so is $H_{m-1,m-1}$

$$\begin{aligned}
 H_{m-1,m-1}^T &= (V_{m-1}^T AV_{m-1})^T \\
 &= V_{m-1}^T A^T (V_{m-1}^T)^T \\
 &= V_{m-1}^T AV_{m-1} \\
 &= H_{m-1,m-1}
 \end{aligned}$$

and because $H_{m-1,m-1}$ is a Hessenberg matrix, $H_{m-1,m-1}$ is a tridiagonal matrix for symmetric A . This is important for the recursive relations of the Conjugate Gradients.

With this orthonormal basis a solution to the linear system would consists of find linear combination, y_m to satisfy $x_m = x_0 + V_m y_m$, with the Ritz-Galerkin condition and (5) this can be done with

$$\begin{aligned}
 0 &= V_m^T (b - Ax_m) \\
 &= V_m^T b - V_m^T A(x_0 + V_m y_m) \\
 &= V_m^T (b - Ax_0) - V_m^T AV_m y_m \\
 &= V_m^T r_0 - H_{m,m} y_m
 \end{aligned}$$

then by $r_0 = \|r_0\|_2 v_1$ and that V_m consists of an orthonormal basis v_i , $V_m^T r_0 = \|r_0\|_2 e_1$, with e_1 being the first canonical unit vector in $\mathbb{R}^m$, we get

$$H_{m,m} y_m = \|r_0\|_2 e_1 \quad (6)$$

$$y_m = \|r_0\|_2 H_{m,m}^{-1} e_1 \quad (7)$$

with this it can also be shown that the residuals r_m is proportional to the basis vector v_{m+1} , [7]. This means that the residuals make up a orthogonal basis for $\mathcal{K}^m(A; r_0)$ and therefore we can rewrite many of the previous equation with the residuals as the basis, R_m , and tridiagonal matrix $T_{m+1,m}$ defined as $H_{m+1,m}$

$$AR_m = R_{m+1}T_{m+1,m} \quad (8)$$

$$R_m^T AR_m = R_m^T R_m T_{m,m} \quad (9)$$

and

$$(10)$$

$$x_m = x_0 + R_m y_m \quad (11)$$

$$y_m = T_{m,m}^{-1} e_1 \quad (12)$$

With all this we can finally derive the recursion relations for CG. The first thing to see is that $R_m^T R_m$ is a diagonal matrix with the squared norms of the residuals on the diagonal, then (9) shows that $T_{m,m}$ can be transformed into by rescaling into positive definite symmetric (PDS) matrix, because A is PDS. This means that $T_{m,m}$ can be LU decomposed without pivoting [8].

$$T_{m,m} = L_m U_m,$$

with L_m being lower bidiagonal and U_m being upper bidiagonal with unit diagonal. Then we can then rewrite (11)

$$x_m = x_0 + (R_m U_m^{-1})(L_m^{-1} e_1) \quad (13)$$

By the fact that bidiagonal matrix are nonsingular if all main diagonal elements are nonzero and a closed form exists for the the inverse [5], the vector $\bar{\alpha} := L_m^{-1} e_1$ can be calculated recursively, with $\delta_0, \delta_1, \dots, \delta_m$ on the main diagonal and $\phi_0, \phi_1, \dots, \phi_{m-2}$ on the sub diagonal of L_m , as

$$\alpha_0 = \frac{1}{\delta_0}$$

$$\alpha_i = -\frac{\phi_{i-1} \alpha_{i-1}}{\delta_i}.$$

Then for $P_m := R_m U_m^{-1} \implies R_m = P_m U_m$ with P_m consists of the vectors $p_0, \dots, p_{m-1}$ as columns and U_m being a upper bidiagonal matrix with unit diagonal and $\beta_0, \dots, \beta_{m-2}$ on the super diagonal, means that the residuals in the columns of R_m can be calculated as

$$r_i = \alpha_{i-1} A p_{i-1} + p_i \quad (14)$$

therefore p_i is defined as

$$p_i = r_i - \beta_{i-1} p_{i-1}. \quad (15)$$

With these recurve relations (13) can be written as, [8],

$$x_m = x_{m-1} + \alpha_{m-1} p_{m-1} \quad (16)$$

with this, (14) and (15) defines the recurrence relations for CG. Only missing the values of α_i and β_i which can be shown to be equal to, [8]

$$\alpha_{i-1} = \frac{\rho_i}{p_{i-1}^T A p_{i-1}}, \quad (17)$$

and

$$\beta_{i-1} = \frac{\rho_i}{\rho_{i-1}}, \quad (18)$$

with

$$\rho_i = r_i^T r_i. \quad (19)$$

With these two calculations we can see the possibility of two zero division problems. For β_{i-1} we have the inner product of $r_{i-1}^T r_{i-1}$, which will equal zero if r_{i-1} is the zero vector. This will not happen in implementation, as the standard stopping condition for CG is when the norm of the newest residual is less than some given small tolerance. As for α_{i-1} , because A is positive definite $(p_{i-1}, p_{i-1})_A = p_{i-1}^T A p_{i-1}$ defines a proper inner product.

2.1.1 Convergens

In exact arithmetic, meaning that there is no rounding, Conjugate Gradient will converges at the latest n iterations for a linear system of size n . This is because the residuals created in each iteration form an orthogonal basis for the Krylov subspace and therefore maximum of n residuals can be made. In implementation this will only occur for well conditioned matrices, meaning a matrix with low condition number. Where the condition number of an invertible matrix can be defined as, [8],

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\lambda_{max}}{\lambda_{min}}. \quad (20)$$

To show a practical converges rate of CG will find a bound for the A -norm of the error at i 'th iteration, $\|x_i - x\|_A$, where x_i is the i 'th approximation and x is the exact solution.

For $u \in x_0 + \mathcal{K}^i(A; r_0)$ and polynomial a Q_i of degree i with $Q_i(0) = 1$ then

$$x - u = Q_i(A)(x - x_0), \quad (21)$$

This is because $u \in x_0 + \mathcal{K}^i(A; r_0) \implies u = x_0 + \hat{Q}_{i-1}(A)r_0$, $r_0 = b - Ax_0$ and $Ax = b$, therefore

$$\begin{aligned} x - u &= x - (x_0 + \hat{Q}_{i-1}(A)r_0) \\ &= x - (x_0 + \hat{Q}_{i-1}(A)(Ax - Ax_0)). \end{aligned}$$

Then define $\tilde{Q}_i(A) = \hat{Q}_{i-1}(A)A$ with $\tilde{Q}_i(0) = 0$

$$\begin{aligned} &= (x - x_0) - \tilde{Q}_i(A)(x - x_0) \\ &= (I - \tilde{Q}_i(A))(x - x_0) \end{aligned}$$

define $Q_i(A) = I - \tilde{Q}_i(A)$, with $Q_i(0) = I$

$$= Q_i(A)(x - x_0).$$

This means that we can write the i th iteration error as

$$x_i - x = P_i(A)(x_0 - x) \quad (22)$$

Denote the eigenvalues and orthonormalized eigenvectors of A as λ_j and z_j , which means we can write r_0 as a linear combination of the eigenvectors $r_0 = \sum_j \gamma_j z_j$ with scalars γ_j and because r_i can be written as polynomial of A times the original residual r_0 , [8], therefore r_i can be written as

$$r_i = P_i(A)r_0 = \sum_j \gamma_j P_i(\lambda_j) z_j. \quad (23)$$

We can now find a bound for the i 'th iterations error

$$\begin{aligned} \|x_i - x\|_A^2 &= \|P_i(A)(x_0 - x)\|_A^2 \\ &= (x_0 - x)^T P_i(A) A P_i(A) (x_0 - x) \\ &= (x_0 - x)^T P_i^T(A) P_i(A) (Ax_0 - Ax) \end{aligned}$$

When by $r_i = b - Ax_i \implies Ax_i = b - r_i$ and $Ax = b$

$$= (x_0 - x)^T P_i^T(A) P_i(A) (-r_0)$$

When by (23)

$$\begin{aligned}
&= (x - x_0)^T P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\
&= (x - x_0)^T A^T A^{-T} P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\
&= (A(x - x_0))^T A^{-T} P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\
&= (r_0)^T P_i^T(A) A^{-T} \sum_j \gamma_j P_i(\lambda_j) z_j
\end{aligned}$$

Again with the transpose of (23)

$$= \sum_l (\gamma_l P_i(\lambda_l) z_l) A^{-T} \sum_j \gamma_j P_i(\lambda_j) z_j$$

Because $Az_l = \lambda_l z_l \implies A^{-1}z_l = \frac{1}{\lambda_l} z_l$

$$= \sum_l \left(\frac{\gamma_l}{\lambda_l} P_i(\lambda_l) z_l \right) \sum_j \gamma_j P_i(\lambda_j) z_j$$

Now because z_j form an orthonormal basis we finally get

$$= \sum_j \frac{\gamma_j}{\lambda_j} P_i(\lambda_j)$$

The sum can now be bounded by the maximum of $|P_i(\lambda_j)|$ and instead of the optimal polynomial P_i using an arbitrary polynomial Q_i of degree i with $Q_i(0) = 1$.

$$\sum_j \frac{\gamma_j}{\lambda_j} P_i(\lambda_j) \leq \max_j |Q_i(\lambda_j)| \sum_j \frac{\gamma_j}{\lambda_j}$$

With similar derivation as the previous, it can be shown that $\sum_j \frac{\gamma_j}{\lambda_j} = \|x_0 - x\|_A^2$.

$$\|x_i - x\|_A^2 \leq \max_j |Q_i(\lambda_j)| \|x_0 - x\|_A^2$$

replacing Q_i with the Chebyshev polynomial C_i transformed and scaled such that when evaluated at zero it equals 1,

$$Q_i(\lambda) = \frac{C_i\left(\frac{2\lambda - (\lambda_{min} + \lambda_{max})}{\lambda_{max} - \lambda_{min}}\right)}{C_i\left(-\frac{\lambda_{min} + \lambda_{max}}{\lambda_{max} - \lambda_{min}}\right)}$$

we get the bound, [8],

$$\|x_i - x\|_A^2 \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^i \|x_0 - x\|_A^2 \quad (24)$$

where $\kappa(A)$ is called the condition number of A and is defined as

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\lambda_{max}}{\lambda_{min}}$$

From this we can conclude that the convergens rate of Conjugate Gradient depends on the condition number of the matrix.

2.1.2 Preconditioning

Because the convergens rate of CG depends on the condition number, if the linear system is transformed such that the new systems condition number is smaller would result in faster convergens. This is called preconditioning. More specific left preconditioning by a matrix M is to transforme the system $Ax = b$ as such

$$M^{-1}Ax = M^{-1}b,$$

with the goal that

$$\kappa(M^{-1}A) \ll \kappa(A).$$

Because $M^{-1}A$ is only symmetric if both are symmetric and they commute, as symmetry is one of the condition for CG to work on the system. This is because a matrix B is symmetric if and only if

$$(Bx, y) = (x, By), \quad \forall x, y \in \mathbb{R}^n.$$

To overcome this is to change the inner product we use in the recursion relations to

$$[x, y] := (x, My). \quad (25)$$

This inner product is a prober inner product if M is SPD. With this inner product $M^{-1}A$ is symmetric

$$\begin{aligned} [M^{-1}Ax, y] &= (M^{-1}Ax, My) \\ &= (Ax, M^{-1}My) \\ &= (x, Ay) \\ &= (x, MM^{-1}Ay) \\ &= [x, M^{-1}Ay]. \end{aligned}$$

The new recurrence relations for CG with $r_0 = M^{-1}(b - Ax_0)$ and $p_0 = w_0$ can be written as

$$\begin{aligned} x_i &= x_{i-1} + \alpha_i p_i \\ r_i &= r_{i-1} - \alpha_i A p_i \\ p_i &= w_{i-1} + \beta_{i-1} p_{i-1} \\ w_i &= M^{-1} r_i \end{aligned}$$

with scalars

$$\begin{aligned} \alpha_i &= \frac{\rho_{i-1}}{p_i^T A p_i} \\ \beta_i &= \frac{\rho_i}{\rho_{i-1}} \\ \rho_i &= r_i^T w_i. \end{aligned}$$

The new relations have an extra matrix vector product compared to the the unpreconditioned system. Because of this it is important that the the decrease in condition number is large enough such that the extra computational cost needed does not make actual run time of the algorithm longer.

An example of a preconditioner is the (point) Jacobi preconditioner, which is constructed from the diagonal of the original system, [2], meaning for matrix $A = (a_{i,j})$ the preconditioner $M = (m_{i,j})$ is defined as

$$m_{i,j} = \begin{cases} a_{i,j} & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \quad (26)$$

This preconditioner has the advantage that it is sparse. It can therefore have fast matrix vector product if constructed as a compressed sparse row matrix (CSR matrix) which has a matrix vector product computational depended on the number of nonzero entries instead of the size of the matrix [1].

2.2 CG Variants

Because of the conditions for Conjugate Gradient that the system matrix and the preconditioner must be symmetric positive definite, we will look at iterative methods derived from CG where these conditions are not needed for the system and preconditioner. This does come with disadvantages for the different variants, for example irregular convergens, increased computational cost and breakdown conditions.

2.2.1 Normal Equation

One way to solve the linear system, $Ax = b$, where the matrix, A , is not symmetric, is to solve an equivalent system, known as the system of the normal equations

$$A^T Ax = A^T b \quad (27)$$

this systems matrix, $A^T A$, is positive definite

$$x^T A^T Ax = \|Ax\|^2 \geq 0, \forall x \in \mathbb{R} \setminus \{0\}, \quad (28)$$

and can only take a value of zero if A is a matrix of all zeros. The new system matrix is also symmetric

$$(A^T A)^T = A^T (A^T)^T = A^T A. \quad (29)$$

Which means the new system can be solve with Conjugate Gradient, but it does not come for free with the extra matrix vector product needed in each iteration of the algorithm, with the transpose of the matrix, which the is difficulty of is depend on the implementation of the matrix.

A bigger problem is the condition number of the new matrix is the square of the original matrix, [7],

$$\kappa(A^T A) = \kappa(A)^2,$$

meaning that convergens will be slower, and with the extra matrix vector product the run time of the algorithm will increase. One could then precondition the system, as described in section 2.1.2, to lower the condition number, this still has the problem of the preconditioner needed to PDS for the Conjugate Gradient to work.

2.2.2 Bi-Conjugate Gradient

Another way to solve a system with unsymmetric A , is with the Petrov-Galerkin approach, [8], which is to find x_m s.t. the residual r_m is orthogonal to another m -dimensional subspace. A choice for this method is subspace is the Krylov subspace generated by A^T and r_0 , $\mathcal{K}^m(A^T; r_0)$, which will lead to the iterative solver known as Bi-Conjugate Gradient, Bi-CG. An elegant way to derive the recurrence relations for this method is to use CG on a specific system, [8],

$$\begin{pmatrix} 0 & A \\ A^T & 0 \end{pmatrix} \begin{pmatrix} \hat{x} \\ x \end{pmatrix} = \begin{pmatrix} b \\ \hat{b} \end{pmatrix}. \quad (30)$$

or $B\tilde{x} = \tilde{b}$ with the preconditioner

$$\hat{M} = \begin{pmatrix} 0 & M \\ M^T & 0 \end{pmatrix}. \quad (31)$$

With this construction CG can be used to solve the system since $\hat{M}^{-1}B$ is symmetric for the inner product $[x, y] := (x, My)$, which is not a proper inner product since $\hat{M}^{-1}B$ is not positive definite, [8]. Because it is not a inner product this shows a problem with Bi-CG, as this can be evaluated to zero when a proper inner product wouldn't, this is known as breakdown of the method.

Which gets us the relations,

$$\begin{aligned} p_i &= M^{-1}r_{i-1} + \beta_{i-1}p_{u-1} \\ \tilde{p}_i &= M^{-T}\tilde{r}_{i-1} + \beta_{i-1}\tilde{p}_{u-1} \\ x_i &= x_{i-1} + \alpha_i p_i \\ r_i &= r_{i-1} - \alpha_i A p_i \\ \tilde{r}_i &= \tilde{r}_{i-1} - \alpha_i A^T \tilde{p}_i \end{aligned}$$

with scalars

$$\begin{aligned} \alpha_i &= \frac{\rho_{i-1}}{\tilde{p}_i^T A p_i} \\ \beta_i &= \frac{(M^{-T}\tilde{r}_{i-1})^T M^{-1}r_{i-1}}{(M^{-T}\tilde{r}_{i-2})^T M^{-1}r_{i-2}}, \\ \rho_{i-1} &= (M^{-T}\tilde{r}_{i-1})^T M^{-1}r_{i-1} \end{aligned}$$

starting residuals $r_0 = b - Ax_0$ and $\tilde{r}_0$ s.t. $(r_0, \tilde{r}_0) \neq 0$, often chosen to be $\tilde{r}_0 = r_0$. With the orthogonality and conjugate, between the two sets of vectors, [7],

$$(r_j, \tilde{r}_i) = 0, \text{ for } j \neq i \quad (32)$$

$$(Ap_j, \tilde{p}_i) = 0, \text{ for } j \neq i. \quad (33)$$

As mentioned breakdown can occur with Bi-CG, other disadvantages with this method is the extra matrix vector products and as with using the system of normal equations is the need to be able to calculate the transpose of the original systems matrix, as said then the difficulty depends on the implementation of the matrix. A more serious problem with Bi-CG and the methods derived from it, described in the next sections, is that in implementation the methods suffer from irregular convergens, even then Bi-CG does work in exact arithmetic, [8]. This means that norm of the current residual may be larger than the previous residual, even if the algorithm does converges.

2.2.3 Conjugate Gradient Squared

Conjugate Gradient Squared, CGS, is a variant of Bi-CG, that was made with main purpose to not have calculate the transpose of the system matrix A , [7]. We will not go through derivation, but will give the idea behind it. The derivations is based on the fact the both sets of residuals in Bi-CG is calculated implicitly as the product of a polynomial and the starting residual, [8],

$$\begin{aligned} r_i &= P_i(A)r_0 \\ \tilde{r}_j &= P_j(A^T)\tilde{r}_0, \end{aligned}$$

and the bi-orthogonality, (32),

$$\begin{aligned} 0 &= (r_j, \tilde{r}_i) \\ &= (P_i(A)r_0, P_j(A)\tilde{r}_0) \\ &= (P_j(A)P_i(A)r_0, \tilde{r}_0). \end{aligned}$$

Then changing to using the residual defined as $\hat{r}_i = P_i^2(A)r_0$. With this the CGS would not have calculate the transpose of the matrix and the calculating the set of secondary residuals, $\tilde{r}_i$, is not needed, [8]. Because we will not use this method, will be explained in the comparison section 2.2.5, we will not show the recursion relations. What would have been shown is that CGS have similar computational cost as Bi-CG, but generally faster convergens. It can still suffer from breakdowns and irregular convergens similar to Bi-CG.

2.2.4 Bi-Conjugate Gradient Stabilized

For the last variant of CG we will be Bi-Conjugate Gradient Stabilized, Bi-CGStab. The derivation uses the same idea as CGS with changing Bi-CG that was look at the bi-orthogonality of the two sets of residuals, and instead of combining the two polynomials into one, $\hat{r}_i = P_i^2(A)r_0$, defining the second polynomial such that it minimizes r_i and such that the recursion relations with the polynomial is simple. For Bi-CGStab the polynomial is of the form, [7],

$$Q_i(X) = (I - \omega_i X)Q_{i-1}(X) \quad (34)$$

the resulting residual is therefore of the form

$$\hat{r}_i = Q_i(A)P_i(A)r_0. \quad (35)$$

To derive the recursion relation for this new residual we will use the polynomial form of r_i and p_i from Bi-CG,

$$r_i = P_i(A)r_0 \quad P_i(A)r_0 = (P_{i-1}(A) - \alpha_i A T_{i-1}(A))r_0 \quad (36)$$

$$p_i = T_{i-1}(A)r_0 \quad T_{i-1}(A)r_0 = (P_{i-1}(A) + \beta_{i-1} T_{i-2}(A))r_0 \quad (37)$$

With this and (34), the recursion relation for the residual can be derived with substitution in (35),

$$Q_i(A)P_i(A)r_0 = (I - \omega_i A)(Q_{i-1}(A)P_{i-1}(A) - \alpha_i A Q_{i-1}(A)T_{i-1}(A))r_0 \quad (38)$$

with the need to derive what the product of $Q_i(A)T_i(A)r_0$. Which can be derived with substitution to get

$$Q_i(A)T_i(A)r_0 = (Q_i(A)P_i(A) + \beta_i(I - \omega_i A)Q_{i-1}(A)T_{i-1}(A))r_0. \quad (39)$$

With $\hat{p}_i = Q_{i-1}(A)T_{i-1}(A)r_0$, the vector form of $\hat{p}_i$ and $\hat{r}_i$ is equal to

$$\hat{p}_i = r_{i-1} + \beta_i(I - \omega_{i-1}A)\hat{p}_{i-1} \quad (40)$$

$$\hat{r}_i = (I - \omega A)(\hat{r}_{i-1} - \alpha_i A \hat{p}) \quad (41)$$

for the relation of x_{i-1} , with $s_i = \hat{r}_i - \alpha_i A \hat{p}$, (41) can be rewritten as

$$\hat{r}_i = \hat{r}_{i-1} - \alpha_{i-1} A \hat{p}_{i-1} - \omega_{i-1} A s_{i-1}. \quad (42)$$

Which gets us the update relation for x_i

$$x_i = x_{i-1} + \alpha_{i-1} \hat{p}_{i-1} + \omega s_{i-1}. \quad (43)$$

Lastly the scalars α_i , β_i , ω_i and ρ_i can be shown to be equal to

$$\alpha_i = \frac{\rho_{i-1}}{\tilde{r}^T A \hat{p}_i} \quad (44)$$

$$\beta_i = \frac{\rho_{i-1} \alpha_{i-1}}{\rho_{i-2} \omega_{i-1}} \quad (45)$$

$$\omega_i = \frac{(As)^T s}{(As)^T As} \quad (46)$$

$$\rho_i = \tilde{r}^T \hat{r}_{i-1} \quad (47)$$

From the orthogonality from the start of the derivation, $P_i(A)r_0, Q_j(A^T)\hat{r}_0 = 0$, says that Bi-CGStab will terminate in exact arithmetic. The whole derivation is based on [8].

2.2.5 Comparison

The computational cost of each iteration for the different method mentioned can be see in table 1. From this it is simple to say that CG is clearly the cheapest of the five methods, but with the condition that the system matrix and the preconditioner are SPD. For the variants of CG we first have CG on the system of normal equations can have 2 extra matrix vector product depending on implementation, but has the problem of squaring the condition number making the converges slower and need of being able to compute the transpose of the matrix. Bi-CG has the similar computational cost without needing the matrix and preconditioner to be SPD and squaring the condition number. But can suffer from breakdowns and has irregular convergens of the residual. CGS does not need the transpose and in practice has faster convergens than Bi-CG, but still suffers from irregular convergens and breakdowns. With the hight computational cost of the five methods is Bi-CGStab with similar convergens to CGS but less irregular convergens compared to it. For the computational cost the most costly is the matrix vector product for which all beside standard CG uses 2 for the system matrix and 2 for the preconditioner, therefore the differences in cost between the variants are similar, [2]. For these reasons we will use Bi-CGStab as the method in our exploration of trying to learn a preconditioner.

Method	Inner product	$\alpha x + y$	Matrix vector product	Precond matrix vector products
CG	2	3	1	1
CG Normal	2	3	1 or (1/1)	1 or (1/1)
Bi-CG	2	5	1/1	1/1
CGS	2	6	2	2
Bi-CGStab	4	6	2	2

Table 1: The computational cost for an iteration for each of the five methods [2]. With columns corresponding to in order the name of the method; number of inner products; number of vector scaling and addition; number of matrix vector products; Number of precondition matrix vector products. a/b indicate matrix vector products with the matrix itself and its transpose respectably. a (b/c) depends on the implementation.

3 Exploration of learning preconditioners

Now that we have the necessary theory about the iterative solver and there convergens. The two first thing that needs to be done in this exploration is to determine the structure of our preconditioner and algorithm that will do the learning of a preconditioner.

3.1 Structure of the preconditioner

The two main structure properties that the preconditioner must have is, first that is easy to apply to a vector. As mentioned in section 2.1.2 the extra matrix vector products that is need to be done in each of the solvers iterations, regardless of the specific solver used, but is more prevalent for the more complicated methods, for example the solver that we will be using Bi-CGStab. The easiest way to *** this the is keep the preconditioner sparse, as then it can be stored as a compressed sparse row. This sparse matrix format has efficient row look up, which is important for fast matrix vector products [1]. Another benefit from limiting to strictly sparse preconditioning matrices is the storage requirement is decreased significantly.

The second is the degrees of freedom most be kept relatively low, as what we are trying to achieve is to learn a preconditioner that lowers the condition number, therefore the iteration count of the solver. The key point is to learn, as the number of degrees of freedom determines how may different preconditioning matrices that can be found. This is a balance between of how fast it can it can explore the space of matrices, which in theory equates to faster learning. Against slower learning but with the possibility of finding a better preconditioner.

The structure we have chosen to achieve these properties is to use an identity diagonal and partition the 1st super diagonal into m as equal sized parts as possible. Because the matrix shift each elements of a vector by the next elements times a coefficient, we will call this preconditioning a partitioned shift preconditioner.

The sparsity of this type of matrix is clear, as it has 2 nonzero elements per row beside the last row with 1 nonzero element. This is also constant for all regardless of the dimension of the system, meaning the matrix vector products should in theory have scaling when working with larger matrix systems. In this construction it is easy to control the degrees of freedom for testing different sizes, as it is equal to the number of partitions

$$\begin{pmatrix} 1 & x_1 & 0 & & & \\ 0 & 1 & x_1 & 0 & & \\ & 0 & 1 & x_2 & 0 & \\ & & & \ddots & \ddots & \\ & & & 0 & 1 & x_{m-1} & 0 \\ & & & & 0 & 1 & x_m \\ & & & & & 0 & 1 \end{pmatrix} \quad (48)$$

To ensure if this preconditioner has any chance of working, we chose four different matrices from Matrix Market [6], which is repository of sparse matrices, and we will precondition these four matrices with 500 different preconditioning matrices, to see if any of the preconditioned systems solver iteration count is less than the system with no preconditioner applied. For all four system the b vector will be a vector of all ones of appropriate size. Each of the four matrices will have slightly different preconditioners, with different number of partitions and different for generating the coefficients for the partitions. The relevant information for both the matrices and their preconditioner can be seen in table 2. The four matrices was chosen such that we no matrix with the same size and condition number. To get consistent results any randomness is seeded and all code is run the uCloud servers, as code ran on different computing configurations can yield different results. This is also to take advantage of parallel computing, as each of the different preconditioned system is independent, meaning each system can be solved with Bi-CGStab independently.

To determine the distribution of partition coefficients, we ran the code on a small subset of the preconditioned systems. This is done because the solver we are using, Bi-CGStab, as described in section 2.2.4 can suffer from breakdowns and is more prevalent when the system becomes ill-conditioned proportional to the size of the system. The starting distribution of this preliminary test was the standard normal. Here very few converged systems and those that did the coefficients of the partitions were generally closer to zero, in order of $|10^{-2}|$ than to ± 1 . From this we concluded that a smaller standard deviation such that the generated values have similar orders of magnitude as this would most likely to have fewer systems that will suffer from breakdowns and converges successfully. For the number of partitions these preliminary test did not give any conclusive results as to the number of partition that gave a better preconditioner. The choice of preconditioner parameters can be seen in table 2

Name	Dimension	Condition number (est)	Number of partitions	Distribution
eris1176	1176	???	15	$N(0, 0.1^2)$
fidap004	1601	$5.17 \cdot 10^3$	15	$N(0, 0.05^2)$
orsirr_1	1030	$1 \cdot 10^2$	10	$N(0, 0.05^2)$
sherman5	3312	$3.9 \cdot 10^5$	10	$N(0, 0.1^2)$

Table 2: Table of the Matrix Market [6] matrices with names; the size of the matrix; the reported condition number; the number of partitions in the super diagonal of the preconditioner; the distribution that generated the coefficients.

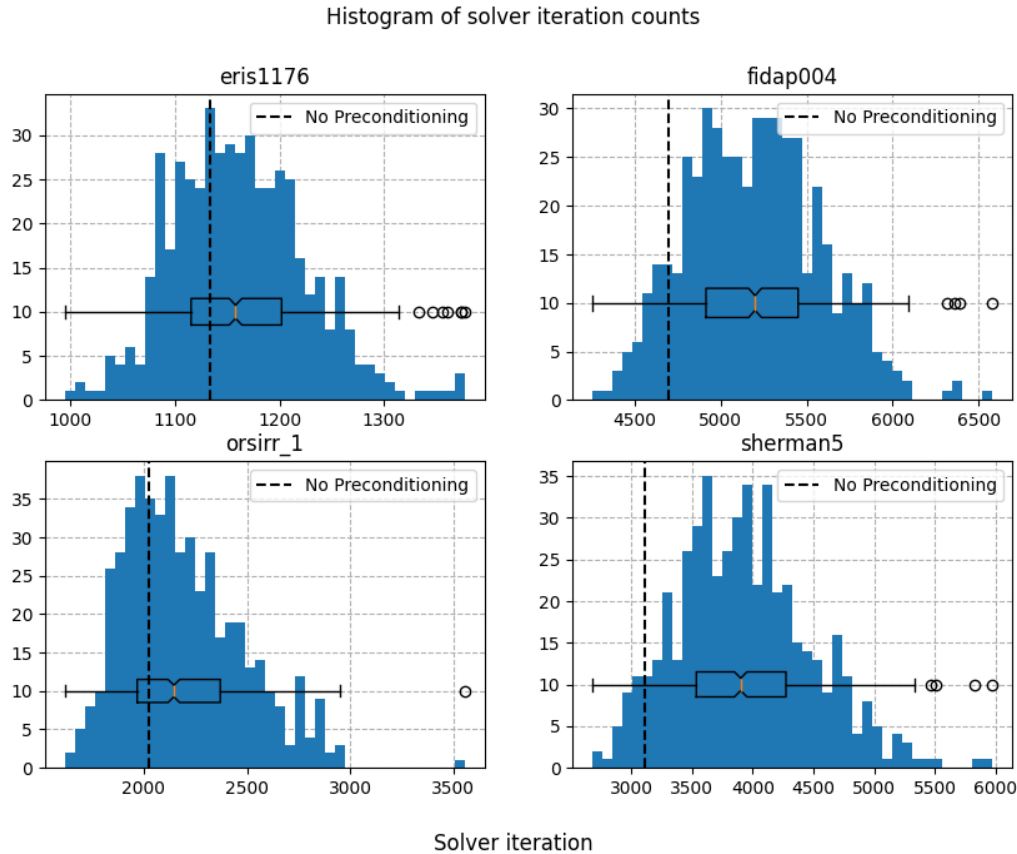


Figure 1: Histogram and boxplot of the solver iterations counts of 500 preconditioned systems for each of the different Matrix Market matrices, with a dashed line corresponding to the unpreconditioned system solver iteration count.

Matrix	No preconditioner	Mean	Median	std	Better	Break down
eris1176	1134	1159.39	1157	81.85	175	1
fidap004	4695	5197.11	5200.5	384.88	47	0
orsirr_1	2025	2190.75	2146	287.80	163	0
sherman5	3115	3904.15	3898.5	606.91	31	5

Table 3: Solver iteration count statics of the four matrices preconditioned systems. With each column in order corresponding to the name of the matrix; the solver iteration count for the unpreconditioned system; the mean, median and standard deviation preconditioned solver iteration counts; the number of preconditioned matrices which have had a solver iteration count lower than the unpreconditioned system; number of preconditioned system that suffered from a breakdown.

From both the histograms in figure 1 and the second to last column of table 3, showing how many of the preconditioned systems had a fewer number of solver iterations, we can see that partitioned shift preconditioner can work with the correct coefficients. And that the condition number of the original system plays a part in how well the preconditioner works, as the two matrices with the highest condition has far the fewest preconditioners that improved the solver iteration count, this is a bit unexpected as system with a higher base solve iteration count has better potential for improvements. It could also mean that the coefficient generations might have a bad distribution to generate a good preconditioner for these two systems. This is

where the learning of the preconditioner comes in, because then the initial coefficient should not matter only if the initial preconditioner would not cause a breakdown of the algorithm to occur.

3.2 Initial learning algorithm

We will now make a learning algorithm that can find a set of coefficients for the partitions of the preconditioner such that the preconditioned system have a decreased solver iteration count. Then we will test the algorithm on the four matrices from the previous section

The approach we take to learn the coefficients of the partition shift preconditioner takes inspiration from stochastic gradient decent. Starting with randomly generated coefficients for the partitions with a predetermined number of partitions. Then choose one of the coefficient at random to be change by value generated from some distribution. With this new changed preconditioner the new system is solved to get a solver iteration count to use as a measure, to determine of the change made to the coefficient had a positive or negative effect on the solver iteration. If it decreases the change is kept, otherwise if it increase the iteration count it is rejected. Regardless of the outcome a new coefficient is chosen to be changed. This procedure is then repeated for a number of predetermined learning iterations. Pseudocode for this algorithm see Algorithm 2.

Algorithm 2 Preconditioner learning algorithm

Require: Matrix A and vector b form $Ax = b$
 $M^{-1} \leftarrow$ Generate initial preconditioner
 $k \leftarrow$ Solver iteration count for $M^{-1}Ax = M^{-1}b$
loop $i = 1, 2, \dots$
 $M_{new}^{-1} \leftarrow$ Changed M^{-1}
 $k_{new} \leftarrow$ Solver iteration count for $M_{new}^{-1}Ax = M_{new}^{-1}b$
 if $k_{new} < k$ **then**
 $k \leftarrow k_{new}$
 $M^{-1} \leftarrow M_{new}^{-1}$
 end if
end loop

The new parameters for the learning algorithm that needs to be decided is number of learning iterations and how to change the partition coefficients. To choose which coefficient to be changed we have decided on uniform distribution for which coefficient to be changed, with the knowledge that for larger number of partitions a larger number of learning iterations is needed to explore each coefficient thoroughly. For the value of the change to the coefficient we will use a uniform distribution with mean of zero and different range for each of the four test matrices. Lastly for the number of learning iterations we set this to 300 for all test matrices. The number of partitions and the distribution parameters can be seen in table 4.

Matrix	Number of partitions	Coefficient Distribution	Change distribution
eris1176	8	$N(0, 0.1^2)$	$Uni(-0.5, 0.5)$
fidap004	8	$N(0, 0.05^2)$	$Uni(-0.1, 0.1)$
orsirr_1	15	$N(0, 0.01^2)$	$Uni(-1, 1)$
sherman5	4	$N(0, 0.05^2)$	$Uni(-0.01, 0.01)$

Table 4: The learn parameters for the different matrices with each column corresponding to in order the name of the matrix; number of partitions in the preconditioner; the distribution for the initial partition coefficient; the distribution for the changing the values of the coefficients.

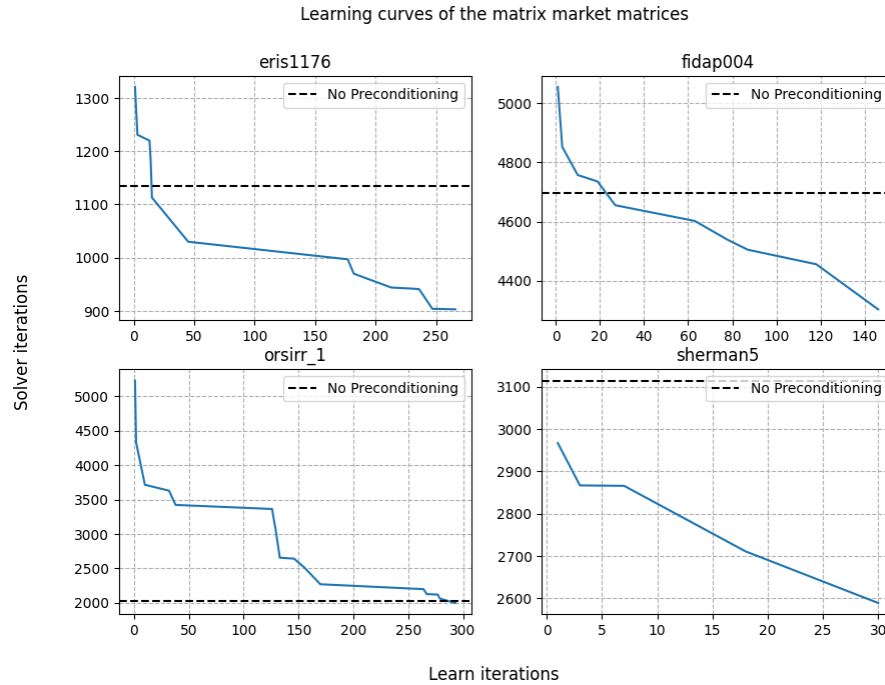


Figure 2: Learning curves of each of the four test matrices. Where the blue line shows the solver iteration count, with data point for each point when the learning algorithm kept a new preconditioner, and a dashed line indicating the unpreconditioned solver iteration count for corresponding matrix.

What immediately can be seen from the learning curve in figure 2, is that our algorithm does what we set out to do as it finds a better preconditioner and that it uses fewer solver iterations than the corresponding unpreconditioned system, the final solve iteration counts can be seen in table 5. Another thing, that is not immediately obvious, is that very few of the iterations the new preconditioner got accepted, at most 17 out of the 300 iterations for orsirr_1 but this may be because of the initial preconditioner making the system much worse. It is hard to tell if this is a real problem because of the fact the all learned preconditioner works better than no preconditioning, even when the initial preconditioner make many times worse, as for orsirr_1. The few number of improvements for sherman5 may be contributed to the fact that the initial preconditioner already improves the system and starts at a lower solver iteration count than the unpreconditioned, so it may have learned what it can very fast. This difficult to say from sherman5's data, but it is clear from the runs for the three other matrices that this learning method can work.

Matrix	Final	Compare v Non	Learning improvement	Number of improvements
eris1176	903	231	418	14
fidap004	4304	391	750	10
orsirr_1	1998	27	3233	17
sherman5	2589	526	378	5

Table 5: Table of data from the learning of the preconditioner with row for each for the matrices and columns corresponding to in order the name of the matrix; the solver iteration count for the final preconditioner; how many fewer solve iterations the preconditioned used compared to the unpreconditioned system; solver iteration count difference between the initial preconditioned system and the final preconditioned system; the number of times a a change to the preconditioned was accepted.

3.2.1 Better exploration of the preconditioner

As mentioned in the previous section the number of learning iterations that had any improvement were quit few. This could be because of a bad exploration of the coefficients in the partitions. The change we will make to the learning algorithm is as such: if a change made to the preconditioner did not improve the solver iteration count, we will try the same change on the same coefficient scaled by 0.5, if that worked the scaled change is accepted and continues as the original algorithm. If this also did not get accepted it will be scaled by 0.25. Then if all scaled changes does not improve the solver iteration count a new change will be generated and a new coefficient is chosen. For pseudocode see Algorithm 3. Because the new version of the learning algorithm will test up to 3 times as many different preconditioners, for a test with this new version to be comparable to the previous test we will use a base learning iteration count of 100 instead of 300. Other parameters of the algorithm will not be changed, table 4.

Algorithm 3 Preconditioner learning algorithm with scaling of the change

Require: Matrix A and vector b form $Ax = b$

$M^{-1} \leftarrow$ Generate initial preconditioner

$k \leftarrow$ Solver iteration count for $M^{-1}Ax = M^{-1}b$

loop $i = 1, 2, \dots$

loop $s = \{1, 0.5, 0.25\}$

$M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s

$k_{new} \leftarrow$ Solver iteration count for $M_{new}^{-1}Ax = M_{new}^{-1}b$

if $k_{new} < k$ **then**

$k \leftarrow k_{new}$

$M^{-1} \leftarrow M_{new}^{-1}$

 Break the scale for loop

end if

end loop

end loop

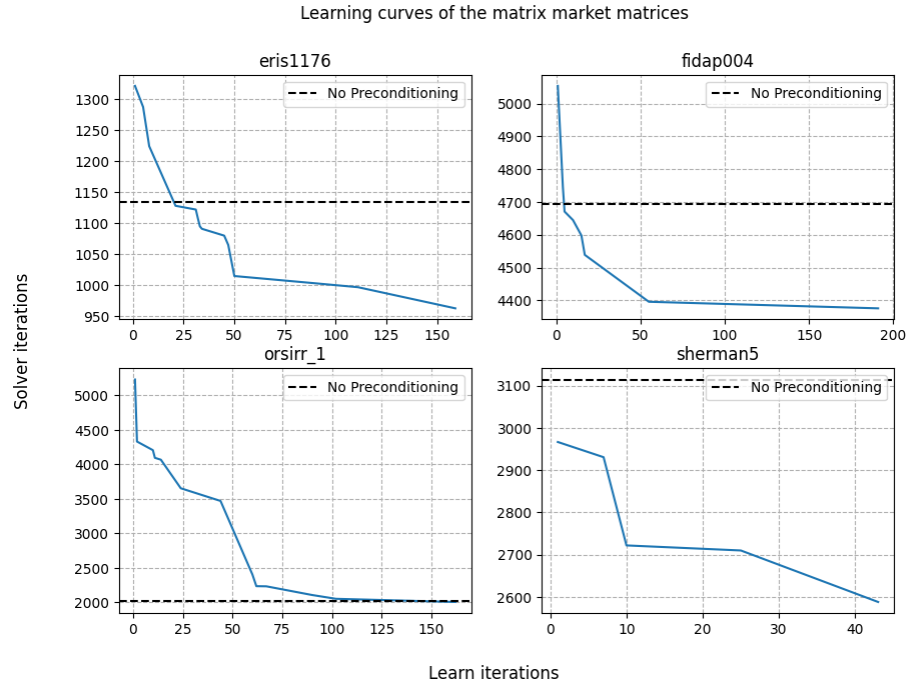


Figure 3: Learning curves with scaling the change during learning for the four different test matrices.***

Matrix	Final	Compare v Non	Learning improvement	Number of improvements
eris1176	963	171	358	14
fidap004	4376	319	678	8
orsirr_1	2007	18	3224	13
sherman5	2588	527	379	5

Table 6: Table of data from the learning of the preconditioner with scaling the change of the preconditioner with a row for each for the matrices and columns corresponds as table 5

When comparing the result to the previous test the learned preconditioner is a small amount worse and the reason for the change to the algorithm also did not improve as the number of accepted preconditioners is also less. A positive improvements is that the learning curves are steeper meaning faster learning, this is especially notable for the matrix fidap004 and might explained the fewer number of improvements. As now the algorithm finds the best preconditioner faster and can therefore find any more changes that would improve the preconditioner. Because the learned preconditioner are only a bit worse and especially the faster learning we will keep the change made to the preconditioner for all the next tests.

3.3 Discrete laplace operator

The main problem with the two previous test and how we are using the learning algorithm, is that we only have one piece of data for each run of the algorithm. A consequences of this is that the algorithm does not learn a preconditioner that could work for some general matrix, but a good preconditioner for that specific matrix, which is what the theses is about. A parallel to this is that with only one matrix the algorithm is unnecessary as we are learning to solve the system with fewer iteration by solving system itself. The only way to solve this is to get more matrices and to have a set matrices for the training part and set to evaluate the learned preconditioner, as to ensure the algorithm is not overfitting to the training data. To do this we will use the 2-dimensional discrete laplace operator, Δ , with periodic boundary conditions, as it has a direct connection to the dirac operator $D^2 = \Delta$.

The D -dimensional discrete laplace operator can be constructed as such [3],

$$\Delta = \sum_{d=1}^D \left(\left(\bigotimes_{k=d}^{d-1} I^{(N_k)} \right) \otimes \Delta_{h_d}^{(N_d)} \otimes \left(\bigotimes_{k=d+1}^D I^{(N_k)} \right) \right). \quad (49)$$

Where $\Delta_{h_d}^{(N_d)}$ is the $N_d \times N_d$ 1-dimensional discrete laplace operator with periodic boundary conditions for the d 'th dimension is defined as

$$\Delta_{h_d}^{(N_d)} = \frac{1}{h_d} \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 & 1 \\ 1 & -2 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \\ 0 & 0 & 0 & \cdots & -2 & 1 \\ 1 & 0 & 0 & \cdots & 1 & -2 \end{bmatrix}, \quad (50)$$

with discretization step size of h_d and N_d grid points. $I^{(N_d)}$ is a $N_d \times N_d$ identity matrix. $A \otimes B$ is the Kronecker product defined between two matrices A and B of size $n \times m$ and $i \times j$ respectively as

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1m}B \\ \vdots & \ddots & \vdots \\ a_{n1}B & \cdots & a_{nm}B \end{bmatrix},$$

and the resulting matrix is of size $ni \times mj$.

The 2-dimensional laplace can then be written as

$$\Delta_{h_1}^{(N_1)} \otimes I^{(N_2)} + I^{(N_1)} \otimes \Delta_{h_2}^{(N_2)}. \quad (51)$$

Because we want the matrices to be invertible, the matrix has to be square and the Kronecker produces a matrix of size $n \cdot i \times m \cdot j$, we will use the same discretization for both dimensions, $N_1 = N_2 = N$ and for ease both of the discretization step sizes will be set to one, $h_1 = h_2 = 1$. An example with $N = 6$ equation (51) evaluates to can be seen in figure 4 as a heatmap, with zeros removed for clarity .

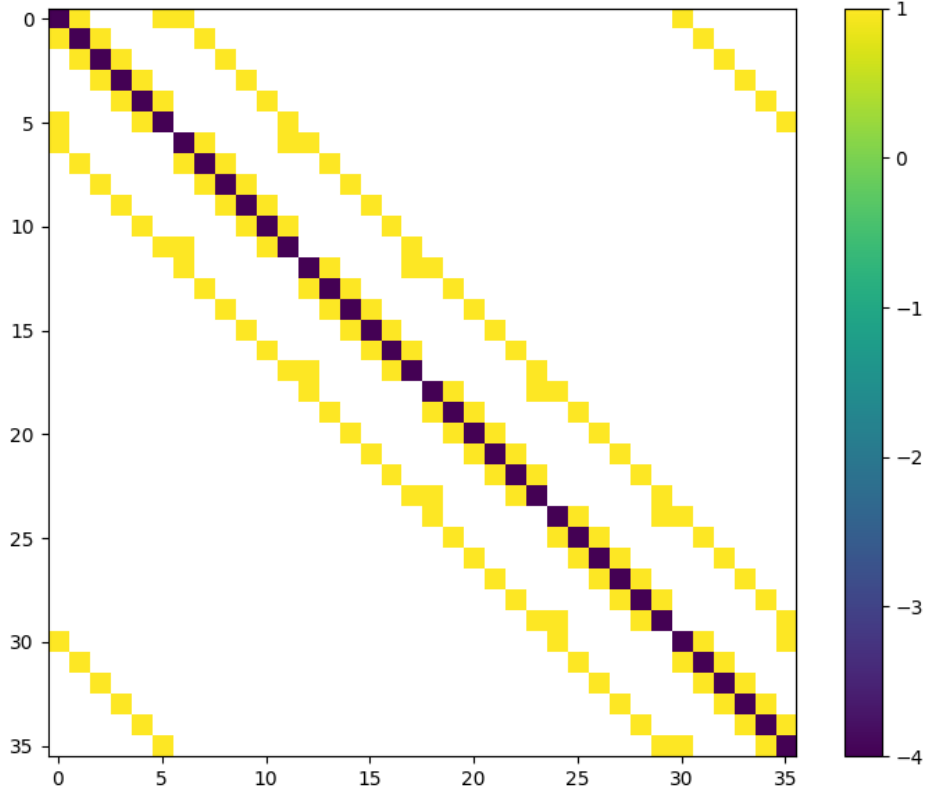


Figure 4: 2-dimensional laplacian operator with discretization of size $N_1 = N_2 = 6$ and discretization step size of 1 for both dimensions show as a heatmap with zeros removed.

Because we want sets of distinct matrices, as we need a unique set of matrices for the training and a set for the evaluate of the learned preconditioner, each nonzero value in the matrices will have some random perturbation applied to it. The simplest way to implement this to perturb the smaller matrices before doing the Kronecker product from equation 51 and because the the discretization is the same for both dimensions there is only needed to create three unique matrices to get unique entries for the 2-dimensional laplace operator, see equation (52). To get small perturbation we use the normal distribution with a mean of 1 and small standard deviation, note that because the perturbations are made to the smaller matrices and the Kronecker product the final perturbations will have a distribution of $N(1, \sigma^2) \cdot N(1, \sigma^2)$.

$$\Delta_p \otimes I_{p_1} + I_{p_2} \otimes \Delta_p. \quad (52)$$

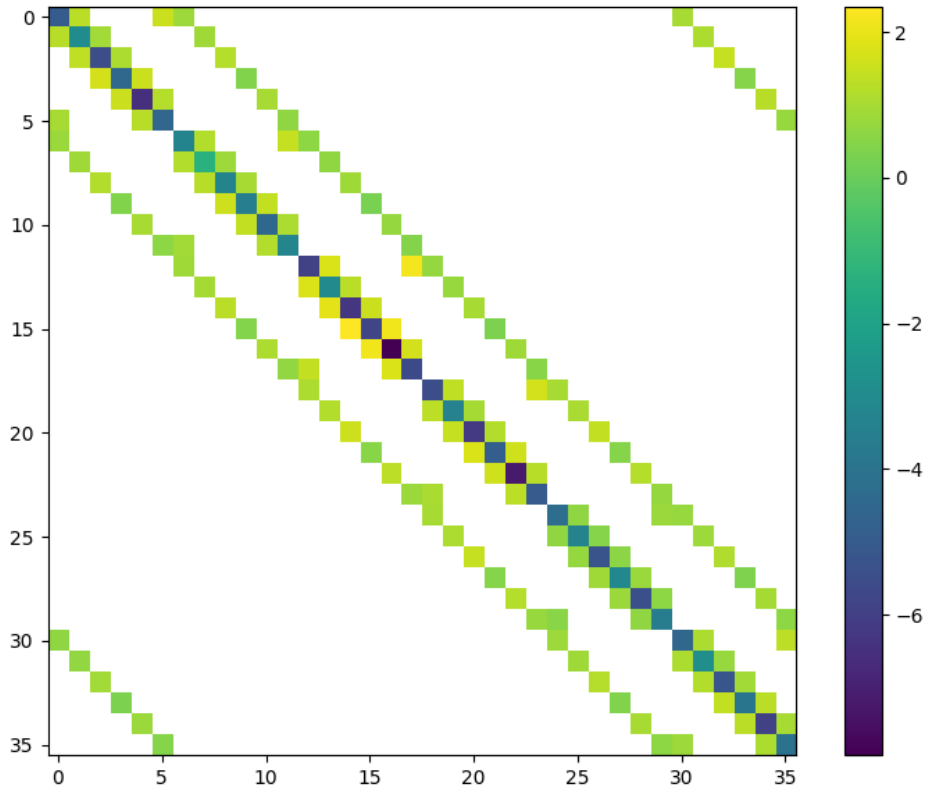


Figure 5: 2-dimensional laplacian operator with discretization of size $N_1 = N_2 = 6$ and discretization step size of 1 for both dimensions with an initial perturbations of $N(1, 0.3^2)$ show as a heatmap with zeros removed.

Now that we can generate matrices, we need to determine the parameters of the data generation. These are the size of the 1-dimensional laplace operators and the the variance of the initial perturbations. What we would like to have for the solver iteration count is that it is larger than the system size, because then the preconditioner has something to work with. On the other hand we don't want the systems to be too ill-conditioned compared to the size of the system, as we do not want breakdowns to occur, or breakdowns in the exploration of the preconditioner. The best way to determine these parameters is to test different configurations and make the decision based on the results. We will test with four different configurations of the parameters with 250 matrices each test set.

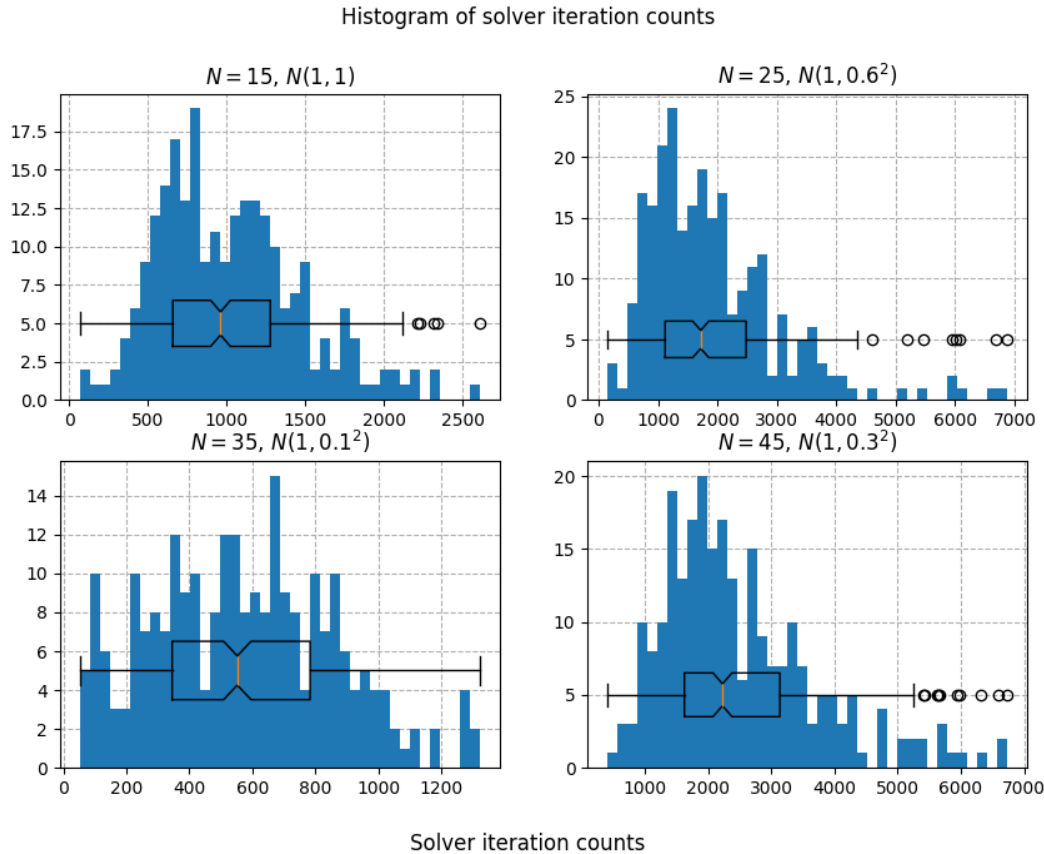


Figure 6: Histograms and boxplot of the solver iteration counts for 4 different configurations of the 2-dimensional discrete laplace operator with perturbations, with breakdown occurrences removed.

N	System Size	Perturbation	Mean	Median	STD	Break downs
15	225	$N(1, 1)$	1019.1	955.5	461.11	17
25	625	$N(1, 0.6^2)$	1927.6	1713	1135.8	2
35	1225	$N(1, 0.1^2)$	567.2	550	290.8	0
45	2025	$N(1, 0.3^2)$	2487.3	2218	1222.4	0

Table 7: parameter configuration and statistic of the four test runs of the 2-dimensional discrete laplace operator with corresponding in order to the size of the 1-dimensional discrete laplace operator; the size of the 2-dimensional discrete laplace operator; the initial perturbation distribution; the mean, median, and standard derivation of the solver iteration; the number of systems where a breakdown occurred.

Histograms of the solver iteration count can be seen in figure 6, and configuration and statistic can be seen in table 7. The first thing to notice from these four tests is the effect of the perturbations variance has on the solve iteration count. It has a positive correlation meaning with larger variance the solve iteration count is increased, to the point of becoming too ill-conditioned compared to the system size, can be seen for configuration with $N = 15$ and perturbation distribution of $N(1, 1)$, but what can not be seen in the histograms, because system that had a breakdown occur are removed for readability of the histogram, is that the breakdown that occurred the latest happened at solver iteration 67337. This is a problem both because that data point is useless for the analysis of the a potential preconditioner, but also because the solving of each of the systems is done in parallel meaning if one of the system take 6 times as long to solve, each

learned iteration has to wait on the that system to be solved before it can continues with the learning. Then if the perturbations are too small, as is the case for the configuration $N = 35$, $N(1, 0.1^2)$, the solver iteration count are also small compared to the size of the system, to the point that a potential preconditioner has very few solver iteration count to improve on. For these reason we will go forward with with the last of the configurations with $N = 45$ and $N(1, 0.3^2)$, as the median solver iteration count is larger than the system size, most iteration count are larger than the system size without having any systems with an occurrences of a breakdown.

3.4 Improvement functions

Now that we can generate data sets of matrices the current version of the learning algorithm needs to be change, as it can not decide when a change made to the preconditioner is an improvement for more than one system. For one piece of data it was simple, is the solver iteration count higher or lower than the system solved with the previous preconditioner. This means we need a measure or function that can determine if the change made to the preconditioner was an improvement for the solver iteration count of all the matrices in the data set, this we will call the improvement function.

As this part of the algorithm is especially important we will test three different improvement functions, for four different seedings of any randomness in the algorithm. Two of the improvement functions are the mean and median of the solver iteration counts and it will accept a change made to the preconditioner if the mean or median solver iteration count is lower than previous preconditioner.

The last improvement function will look at the individual changes of the solver iteration counts and check if more have a decreased iteration count than an increased iteration count. In implementation this is done by entry wise difference between the unchanged preconditioned systems and the changed preconditioned system. Then counting how many have a negative sign and a positive sign and if more have a positive sign the change to the preconditioner is accepted if not the change is rejected, for this reason this improvement function will be called the sign improvement function. More generally this means a preconditioner is accepted if more than half of the individual iteration count have decreased.

One problem that might occur with the sign improvement function is: lets say on a data set with 100 matrix systems, 55 of them have a decrease in solver iteration count of 1 with the new preconditioner, and the last 45 have in increase of 1000 iterations with this new preconditioner. This state would be accepted and these new iteration counts will both have larger mean and median, therefore not be accepted by the other to improvement functions. How likely it is for it to occur and if the problem is a big as given example, is to be determined by the testing of the improvement function.

Another point to consider is what to do when change to the preconditioner makes one or more of the systems have a break down occur, as this would drastically change the solver iteration count of the system in both directions and skew all three of the improvement functions measure. We could remove the data point from the improvement measure, but this would still change result of the improvement function. Because if system with previous largest iteration count had a breakdown the mean and median would be smaller and for the sign improvement function ***. For these reasons we will also add the condition that all preconditioned systems must converge for a change to the preconditioner to be accepted, this could cause problem if a system starts ill-conditioned. The new version of the algorithm see algorithm 4

N	Perturbation	Amount	Base learn iterations	Learn Scales	Initial coefficient distribution	Number of Partitions
45	$N(1, 0.3^2)$	250	100	$\pm\{1, 0.5, 0.25\}$	$N(0, 0.05^2)$	10

Table 8: Run parameter configuration in order each column corresponds to the size of the 1-dimensional laplace operator; the initial perturbations; the amount of matrices in each data set; The base number of learning iteration; the scales each change will attempt with; the initial coefficient distribution; the number of partition in the preconditioner.

Algorithm 4 Preconditioner learning algorithm with improvement functions

Require: A set of matrices and vectors than each form a linear system $A_i x = b_i$

Require: An improvement function f

$M^{-1} \leftarrow$ Generate initial preconditioner

$k_list \leftarrow$ A list of solver iteration count for each $M^{-1} A_i x = M^{-1} b_i$

loop $i = 1, 2, \dots$

loop $s \in \pm\{1, 0.5, 0.25\}$

$M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s

$k_list_{new} \leftarrow$ A list of solver iteration count for each $M_{new}^{-1} A_i x = M_{new}^{-1} b_i$

if f accepts the new state and all systems converged **then**

$k_list \leftarrow k_list_{new}$

$M^{-1} \leftarrow M_{new}^{-1}$

 Break the scale for loop

end if

end loop

end loop

The other run parameters, see table 8, will be the same for all 12 test. The choice of the parameters are made from the conclusions form the previous sections tests. With a change to the list of scales $\pm\{1, 0.5, 0.25\}$ as to explore both directions of the possible change to the coefficient.

Starting with median as the improvement function we see from its learning curves in figure 7, that it had hard time finding any changes to the preconditioner that would improve the median solver iteration count with at most 12 accepted preconditioners for seeding 0 and down to 3 accepted for seeding 3. Table 9 for all acceptance counts. Because of the few number acceptances the evaluation of the preconditioner on the test data set preforms the worst for all three measures compared to two other improvement functions for all seedings. For the mean and median solver iteration count compared to the unpreconditioned systems the values are very similar, figure 8, and only about a fourth of the system have a lower iteration count compared to the unpreconditioned systems, figure 9. With this we can say that the using the median as the improvement function in learning does not work.

With mean as improvement function this problem is not as prevalent, but still happened for seed 0 with 9 instances of improvement occurring. Of the four runs with mean as the improvement function this seeding is the one that worst in the evaluations of the learned preconditioner of the four, while the individual comparison almost a three fourths of the systems have a lower iteration count compared to the unpreconditioned systems. For the other three seedings the learned preconditioners does decrease the solver iteration counts. Lastly with sign as improvement function does not suffer from the problem with few iterations of improvement, and therefore consistently have a better evaluation of the learned preconditioners but the fluctuations of the learning curve is because of the problem described earlier. From this we can conclude that a preconditioner with this bidiagonal structure can be learned, with the right improvement function, and that it can work better than using no preconditioner.

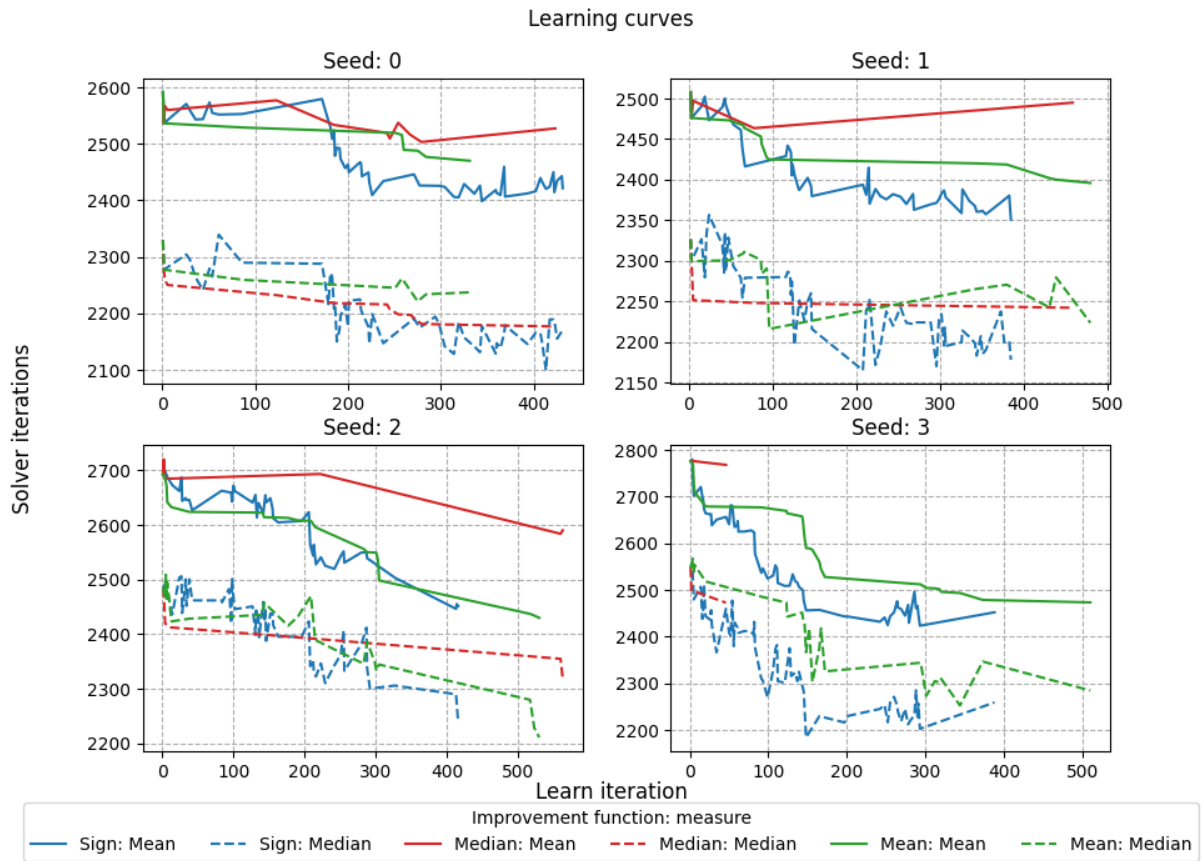


Figure 7: Learning curves for all 12 run configurations. Each plot corresponds to a different seeding. The blue lines used the sign improvement function, red lines used the median as the improvement function and green lines used the mean as the improvement function. The full lines is the mean solver iteration count in each learning iteration and a dashed is the median solver iteration count.

Seed	Sign	Median	Mean
0	56	12	9
1	60	5	14
2	52	7	22
3	64	3	25

Table 9: Number of times a new preconditioner was accept. A row for each seeding and column for each improvement function.

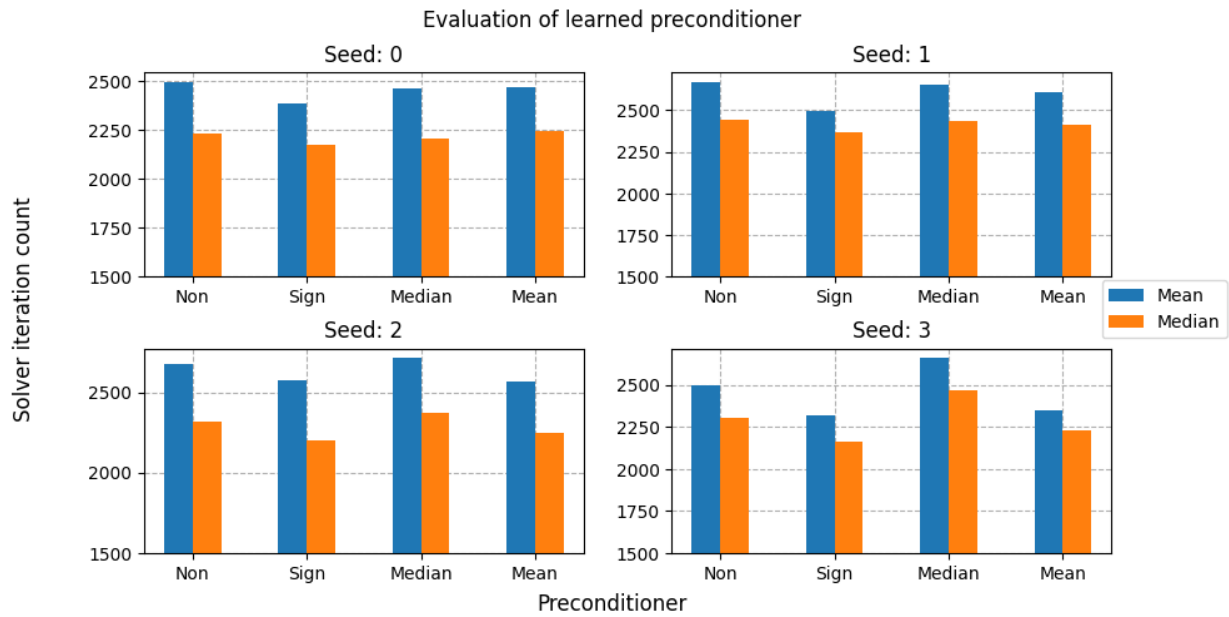


Figure 8: Comparison plot of the evaluation of the learned preconditioner where each plot corresponds to a different seeding in the learning. The colour of the bars corresponds to the mean solver iteration count for blue and median solve iteration count for orange. The groups is the different improvement functions used in learning which are in order the unpreconditioned system; sign improvement function; median as the improvement function; mean as the improvement function. Lower is better.

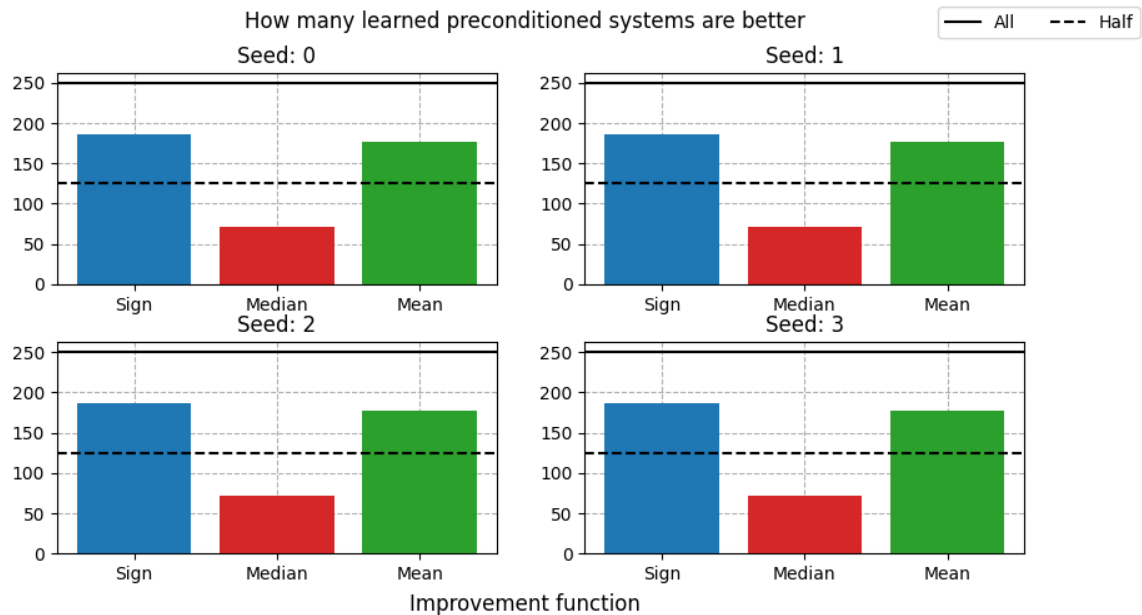


Figure 9: Bar plot of of many of the system where the preconditioner improved the solver iteration count compared to the unpreconditioned systems. Each plot is for a different seeding. The bars corresponds to a different improvement function used in learning indicated by its label. The dashed line is half the data set and full is the whole data set. Higher is better.

3.4.1 A better comparison

For a better comparison of the learned preconditioners, the evaluations should be compared against already know working preconditioner. Because we have information about the structure of the matrices we want to precondition, that is the largest absolute value are on the diagonal and these values do not equal to 1 or 0. Therefore a good choice of preconditioner is the Jacobi preconditioner, as mentioned in section 2.1.2, which uses the diagonal elements of a matrix as the preconditioner, $\text{diag}(a_{11}^{-1}, a_{22}^{-1}, \dots)$ for matrix $A = (a_{i,j})$ and therefore a trivial to compute inverse. Preconditioning the test data with the Jacobi preconditioner and using the these solver iteration counts as the point of comparison for the learned preconditioner.

What we get from that is the Jacobi preconditioner is better than all learned preconditioners regardless of improvement function and seeding, as the mean and median solver iteration count is lower for all four seedings, figure 10. On the individual matrix level comparison there are systems where the learned preconditioner has lower solver iteration counts, at most 44 systems for seeding 1 and sign as improvement function, figure 11. We conclude from this is that our learned preconditioner can still work, but that the current algorithm or preconditioner needs to be change in some way.

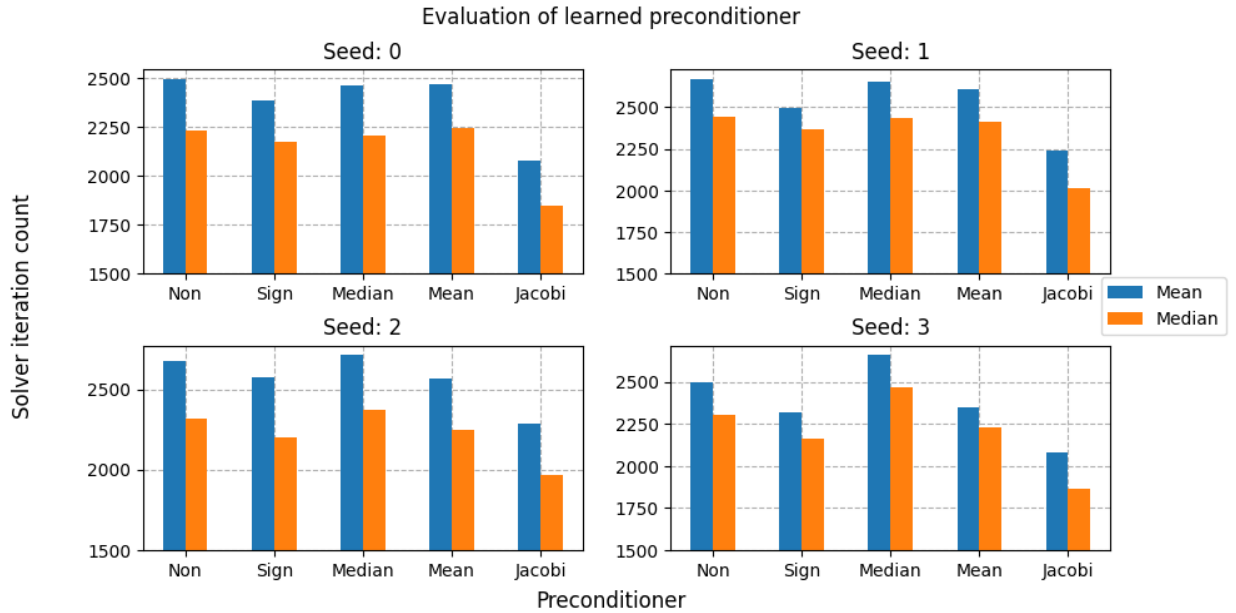


Figure 10: The same plot as figure 8 with a bar added for with the systems preconditioned with Jacobi preconditioner.

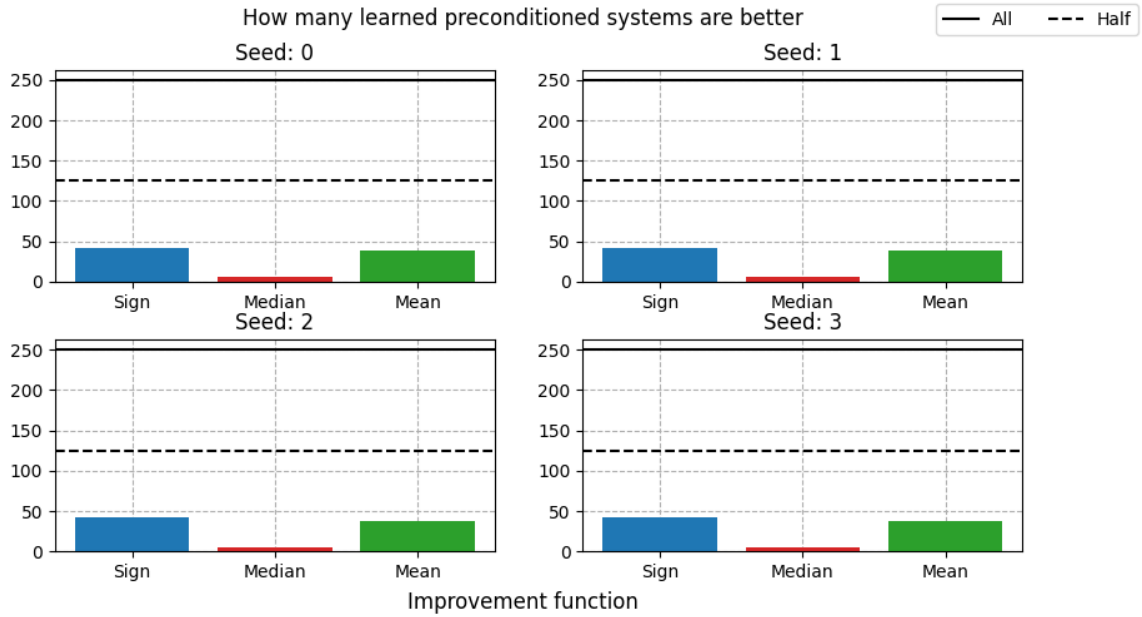


Figure 11: A similar plot to figure 9 with the difference of instead of comparing the solver iteration count to the unpreconditioned system it is compared to the systems preconditioned with Jacobi preconditioner.

3.5 New type (Jacobi shift)***

The change we will make is to the structure of the partition shift preconditioner. That change is replacing the diagonal of the matrix from the identity to inverse of the diagonal of the matrix it is applied to. Meaning combining Jacobi preconditioning and our partition shift preconditioner to get a new preconditioner we will call Jacobi partitioned shift preconditioning. This means our new preconditioner consistent of matrix specific part, the main diagonal, and a general learned part, the partitioned super diagonal. A constraint that the new preconditioner has for the matrix it is applied to is that all diagonals of the matrix has to have a nonzero value.

To test this new preconditioner we will run the learning algorithm with the same parameters as the previous test in section 3.4 table 8, with the only change being the preconditioner.

Seed	Sign	Median	Mean
0	39	25	29
1	37	27	23
2	58	27	26
3	65	37	34

Table 10: Number of times a new preconditioner was accept. A row for each seeding and column for each improvement function.

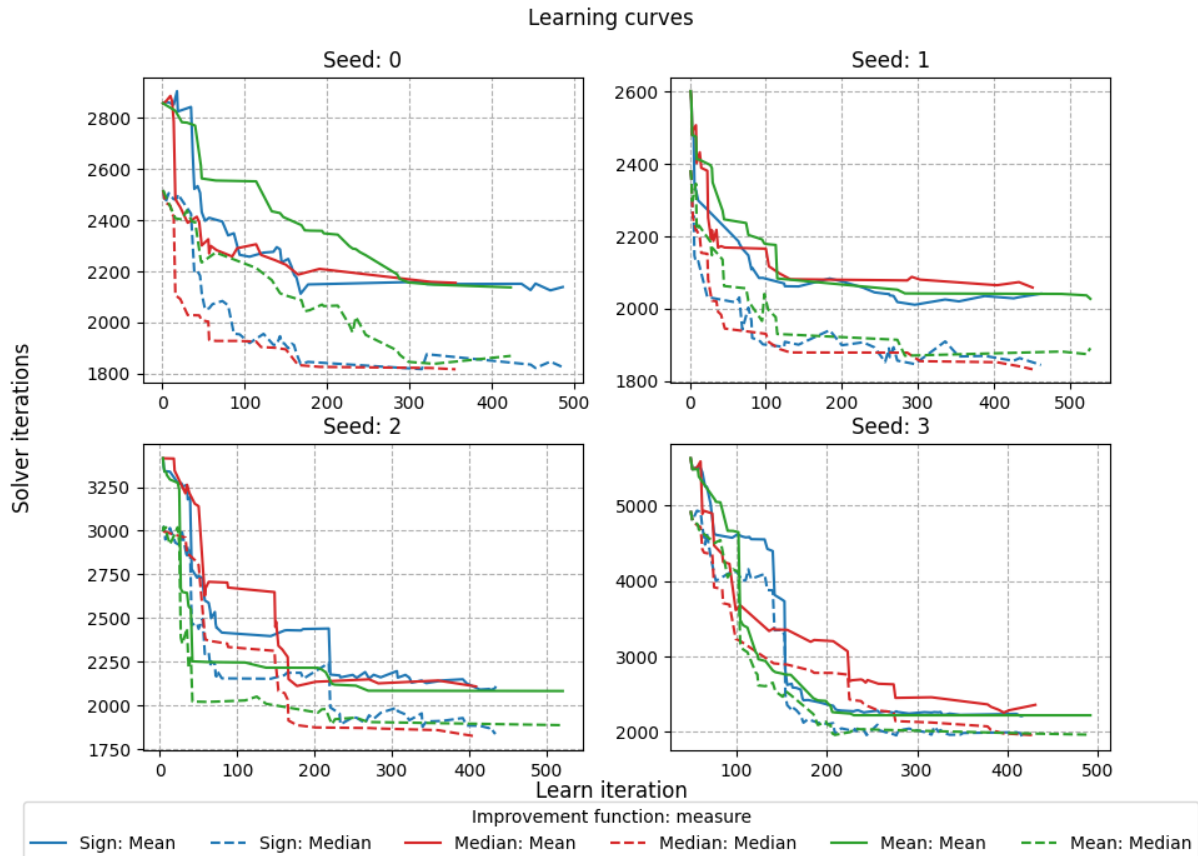


Figure 12: Learning curves for the test runs of with the Jacobi partition shift preconditioning. The plot has the same structure as figure 7, a plot for each seeding, colours indicate improvement function, and line style corresponds to solver iteration count mean or median.

From the learning curves in figure 12, we can see clear improvements when comparing to the learning curves from the previous tests in figure 7. The mean and median solver iteration count decrease significant amount during the learning for all three improvement function, where before with median improvement function had difficulty finding improvements to the preconditioner. Even when the initial preconditioner starts with mean solver iteration count above 5000 for seeding 3 and the final iteration counts is only a bit higher compared to the other seedings, but this may be because the data set and not the preconditioner.

Another improvement is the number of accepted coefficient changes made during learning is considerably larger for mean and median as improvement functions and similar for sign, table 10, again comparing to the learning of the previous tests in section 3.1, this means that a good preconditioner is being learned. And lastly from the learning curves it looks like the learning has plateaued, example for seeding 1 the amount of change that happens to the mean and median solver iteration count after learning iteration 150 is minimal

compared to the earlier leaning iterations. This could mean that it has found the best preconditioner, that it has found a local minimum and the changes made to the preconditioner are not big enough to find a better minimum, or that it did not have enough learning iteration to find a better preconditioner.

As for the evaluations of the preconditioner on the test data, there is only a marginal improvement compared to Jacobi preconditioning on its own for the mean and median solver iteration counts, figure 13, and only with sign and mean as improvement functions we get more than half of the systems having a lower solver iteration count than when using Jacobi preconditioning, figure 14. For the differences in improvement function there is very little difference in the evaluations of the learned preconditioners, with Sign being slightly better all across the seedings for this reason and because it worked the best in the previous test we will only use the sign improvement function for all further testing of the learning algorithm and structural changes to the preconditioner. From this it is easy to say that these learned preconditioners with Jacobi diagonal are better than the previous learned preconditioners with identity diagonal, but only slightly better than the simple Jacobi preconditioning.

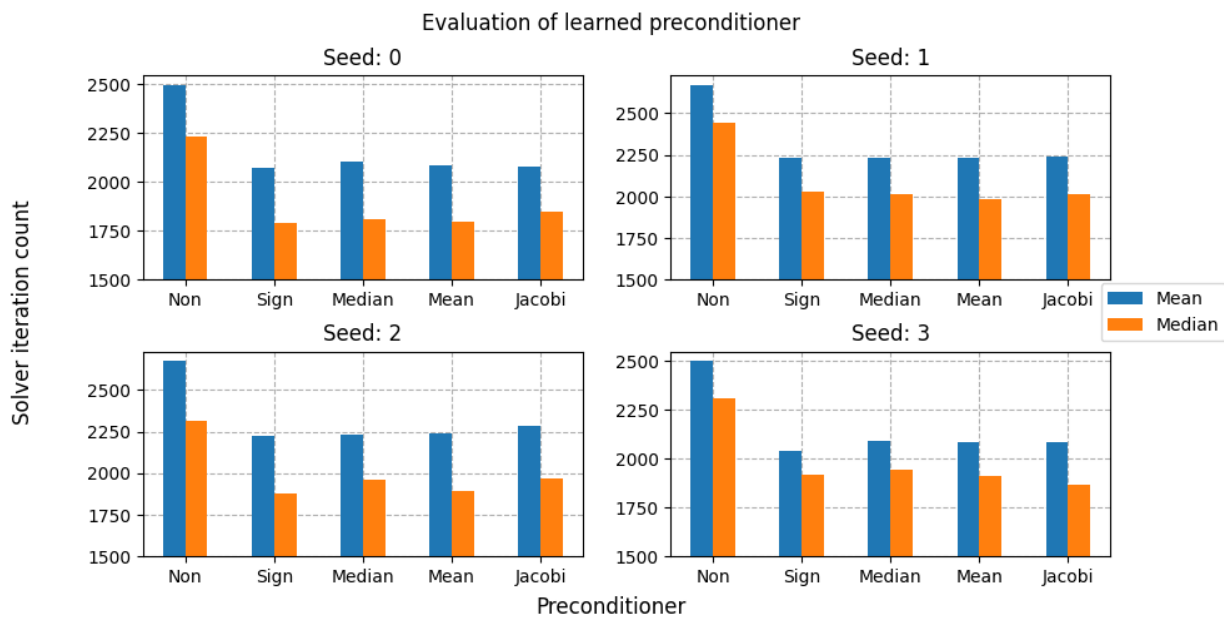


Figure 13: Evaluation of the learned preconditioners with the same structure as figure 10. A plot for each seeding, with a bars for the three improvement functions and 2 to compare with, with unpreconditioned and Jacobi preconditioned data. Bar colours indicate mean, blue, and median, orange, solver iteration count.

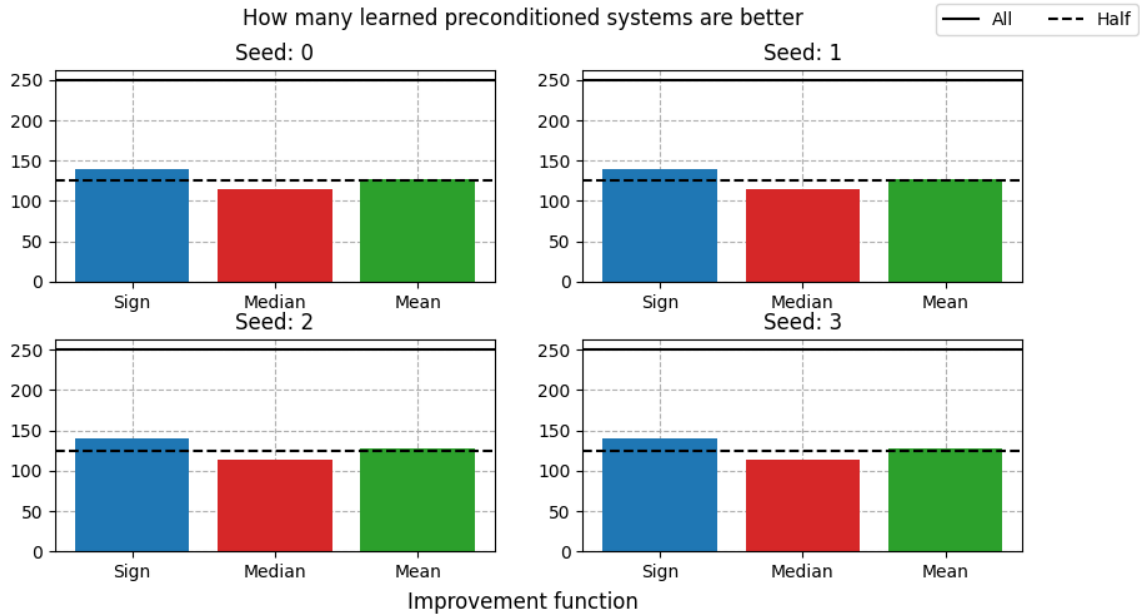


Figure 14: individual system comparing plot with the same structure as 11. A plot for each seeding, bars for each improvement function, and lines to indicate all or half the systems.

3.5.1 Plateau in learning

As mentioned the previous tests learning curves seemed to have plateaued. Therefore we will run the configuration from the previous test with seeding 0 and sign as the improvement function. This is to see if the learned preconditioner is in a minimum, if that is the case the solver iteration counts should stay relatively unchanged. Or if the algorithm could find a better preconditioner after the base 100 learning iterations from the previous test.

What we get from the learning curve in figure 15 is that the learning has plateaued and no significant changes are made after 500th real learning iteration. The fluctuations is a problem inherent to the sign improvement function, which has been mentioned earlier. In the evaluation of the learned preconditioner results, excepted from the learning curve, no significant differences between the run with 100 and 500 base learning iterations, having only slightly higher mean and median solver iteration counts, this is because in the current version of the algorithm the learned preconditioner is the last version in learning, meaning because of the fluctuations it ended in with a slightly worse preconditioner.

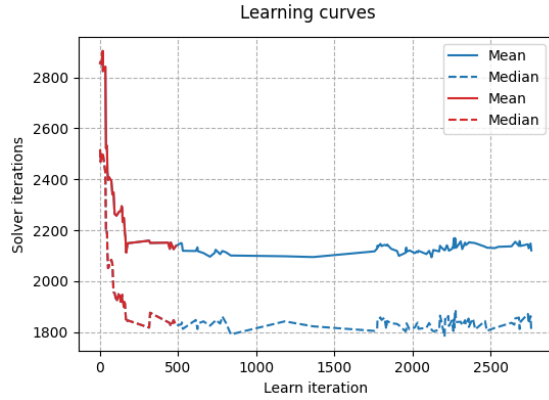


Figure 15: Learning curve for seeding 0 and sign as improvement function with 100 base learning iterations, red curve, and 500 base learning iterations, blue curve. Line style indicate mean, full line, or median, dashed line, solver iteration count.

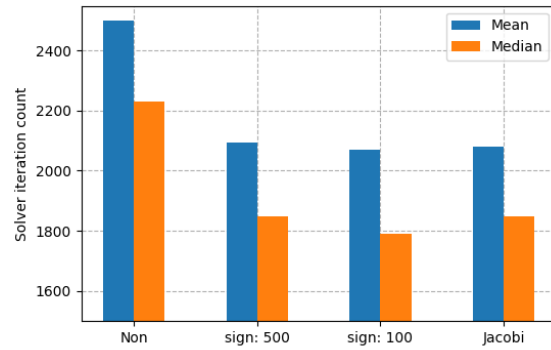


Figure 16: Evaluation of the learned preconditioner, with the same structure as figure 13, for the run with seeding 0 and sign as improvement function with 100 and 500 base learning iterations.

3.6 Allowing disimprovement

To try overcoming the problem of the algorithm plateauing the learning, we will allow changes made to the coefficients to be accepted, even when the improvement function would have decided to rejected the change. We do this because we suspect that the preconditioner is an minimum, suspect a local minimum, and hope by accepting these changes to find a better minimum. The acceptance of a worse state will be done be with a probability that is inverse proportional to how much worse the change is making the new preconditioner, as to not accepted every change made. Where worse is defined by the improvement function.

Meaning with the Sign improvement function for a total of 100 matrices: if 45 preconditioned systems solver iteration count decreased and 55 increased it will have a acceptance probability of $\frac{1}{(55-45)+2} = \frac{1}{12}$. The +2 is there for two reasons, first a +1 to ensure no zero division when an equal amount of system have a decrease and increase in solver iteration count, and second another +1 is there such that not all of these changes where this happens will be accepted. This implementation might need adjustment because the probability is also proportional to the total number of matrices. If the data set is small the probability of accepting a state where all system solver iterations count increase would be larger than if the data set was bigger. There is still the condition that all system must converge for any state to be accepted.

Algorithm 5 Preconditioner learning algorithm with with disimprovement**Require:** A set of matrices and vectors than each form a linear system $A_i x = b_i$ **Require:** An improvement function f $M^{-1} \leftarrow$ Generate initial preconditioner $k_list \leftarrow$ A list of solver iteration count for each $M^{-1} A_i x = M^{-1} b_i$ **loop** $i = 1, 2, \dots$ **loop** $s \in \pm \{1, 0.5, 0.25\}$ $M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s $k_list_{new} \leftarrow$ A list of solver iteration count for each $M_{new}^{-1} A_i x = M_{new}^{-1} b_i$ **if** all systems converged **then****if** f accepts the new state or with probability accept a worse state **then** $k_list \leftarrow k_list_{new}$ $M^{-1} \leftarrow M_{new}^{-1}$

Break the scale for loop

end if**end if****end loop****end loop**

To test this change run the same configuration as the runs form the previous section With this change made to the learning algorithm we ran the same configuration as the as in section 3.5.1, Sign as improvement and 500 base learning iterations.

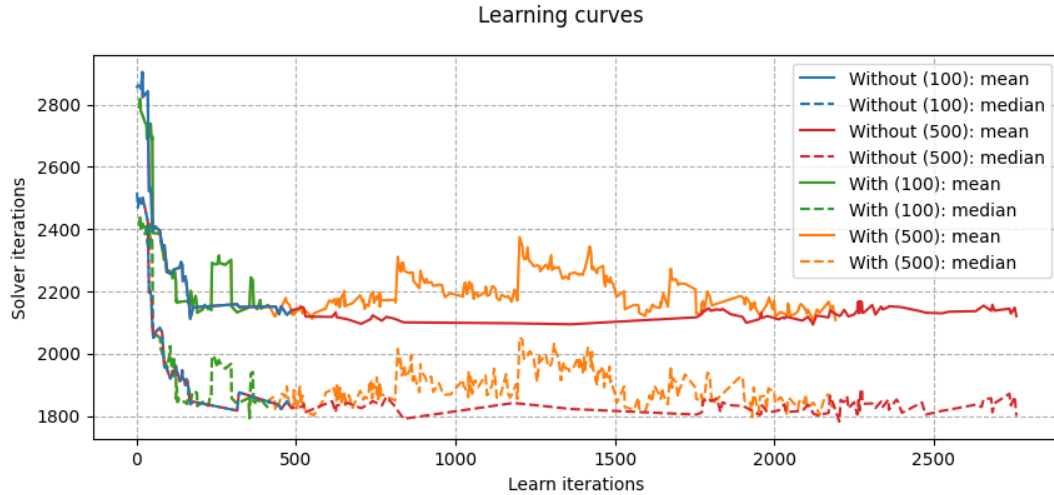


Figure 17: Learning curves for the testing disimprovement where the blue and red lines corresponds to runs without disimprovement with 100 and 500 base learning iterations, this is the same data as in figure 15, green and orange lines are with disimprovement with 100 and 500 base learning iterations, the new test. Line style corresponds to whole line for the mean and dashed line for the median solver iteration count.***(data in plot is wrong without 100 and 500 are not equal)

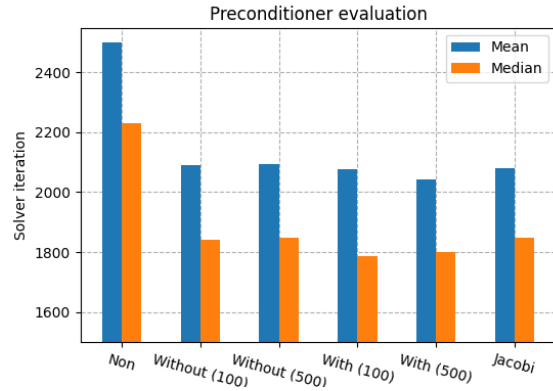


Figure 18: Mean and median solver iteration count for the evaluation of the learned preconditioner. Blue bar is mean and orange is median. A bar group for each of the four tests: without disimprovement with 100 and 500 base learning iterations and with disimprovement with 100 and 500 base learning iteration. On either end the unpreconditioned and Jacobi preconditioned systems for comparison.

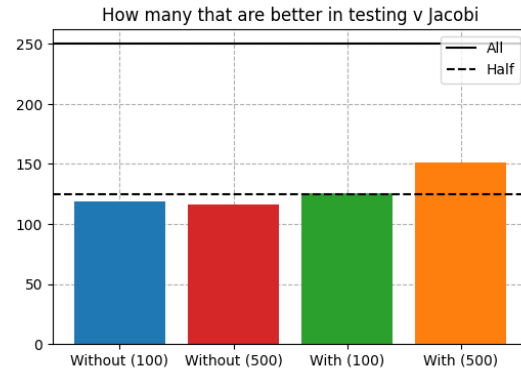


Figure 19: Individual linear system comparison between the learned preconditioners and Jacobi preconditioners with bar size corresponding to the number of systems where the learned preconditioner has a lower solver iteration count than Jacobi Preconditioning.

From the results of this test we get a marginal improvement of the learned preconditioners, meaning we will keep it allowing disimprovements. A thing that was overlooked in the implementation of the disimprovement in the learning algorithm, is that the resulting preconditioner comes from the last accepted learning iteration, meaning that the best found preconditioner is most likely not the preconditioners the algorithm gives as the output. For example if the learning had stopped at solver iteration count spike at learning iteration around 1250, you could naively conclude that allowing disimprovement is worse

To resolve this problem the algorithm should keep tack of the best seen preconditioner. Here we would like to again use the improvement function as a measure for which were the best, but the sign improvement function can not do comparisons across multiple solver iteration count data sets. Therefore we will use the discarded improvement functions mean and median in combination to determine which accepted preconditioner was the best. This measure of best will be done after a new preconditioner have been accepted.

3.7 Number of partitions

Because of the results from the previous test, allowing disimprovement, did not improve the learned preconditioner by any significant degree, we will w go back to looking at the structure of our preconditioner. When initial explaining the structure of the Preconditioner, in section 3.1, in the part about the number of partitions of the super diagonal. We said that with higher number of partitions could lead to a better preconditioner, but with a longer learning time. Therefore the next test will test for larger amount of partitions and the number of partitions we will test is $\{15, 20, 25, 30, 40, 50\}$. To compensate for the potential longer learning time these test will also have an increased base number of learning iteration of 200 instead of the previous 100.

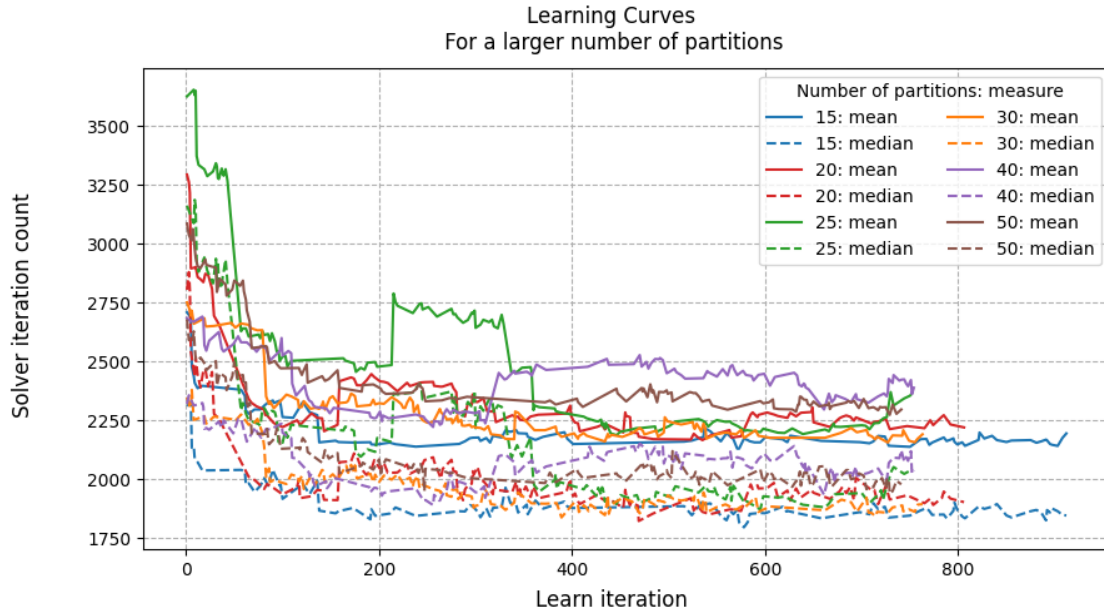


Figure 20: Learning curves for for testing larger number of partitions. Where colour corresponds to the number of partitions and line style corresponds to mean, whole line, or median, dashed line, solver iteration count.

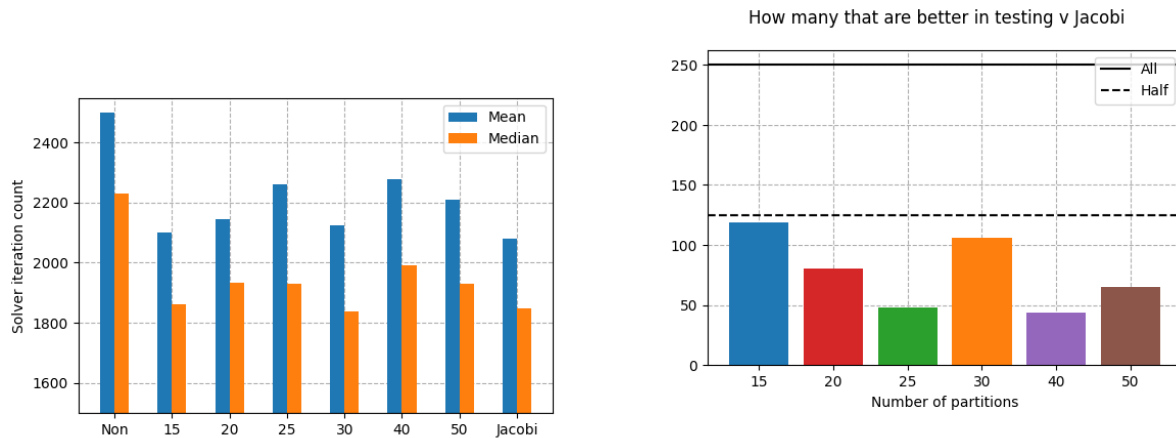


Figure 21: Mean and median solver iteration from the evaluation of the learned preconditioners with different number of partitions indicated by the label.

Figure 22: Individual linear system comparison between the learned preconditioners with different number of partitions and Jacobi preconditioners with bar size corresponding to the number of systems where the learned preconditioner has a lower solver iteration count.

The first thing to notice from the learning curves in figure 20, is that our assumption that having more partitions will equate to longer learning. It can most notable be seen in the test with 50 partition, brown learning curve, as the mean and median solver iteration count does go down but at a slower rate compared to the smaller number of partitions. Starting with a mean solver iteration count of 3086 and decreases it to 2332.3 at learn iteration 248, but then the solver iteration counts does not change by any substantial amount after than. Compared to the run with 15 partitions gets below a mean solver iteration count 2250 at learn iteration 138, which means that more partitions have slower learning and it plateaus at higher solver iteration counts.

The second thing is that allowing accepting of worse state makes the learning worse for higher partition counts. It can most notably be seen for the 40 partition count run, purple learning curve. At learn iteration 324 the mean solver iteration count increases from 2238.3 to 2424.8 and it does not get below the increase before it the number of learning iterations maxed out. Compare this with the run with 25 partitions, which has increase in solver iteration count at learn iteration 215 and does get back to the iteration from before the increase at learn iteration 337. This problem could be improved with a smaller probability for accepting worse states, but then the learning might get stuck in a minimum.

From the evaluations of the learned Preconditioners, figure 21 and 22, it is clear that using more partitions does not make the preconditioner better, even with the change to the algorithm that returns the best found preconditioner during learning, as both the mean and median solver iteration count are on par or worse than using the Jacobi Preconditioning. And all runs the number of systems that have a lower solver iteration count compared to Jacobi is less than half.

3.7.1 Fewer

Because using more partitions did not improvement the learned preconditioners, we will now test with fewer. The partition count we will use in this test is $\{1, 2, 3, 4, 5\}$ and that is the only difference between the runs. With the 200 base learning iterations to make the results comparable with the previous test with more partitions.

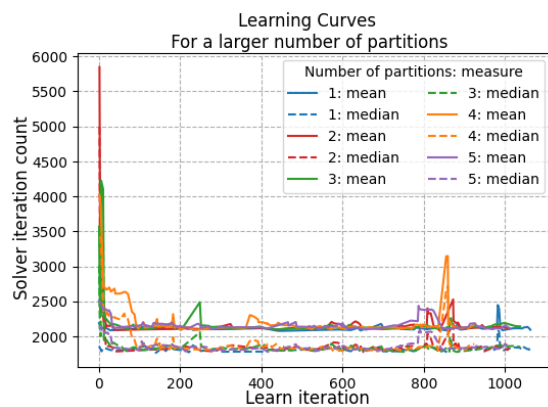


Figure 23: Learning curves for the testing with fewer number of partition, with the same structure as figure 20.

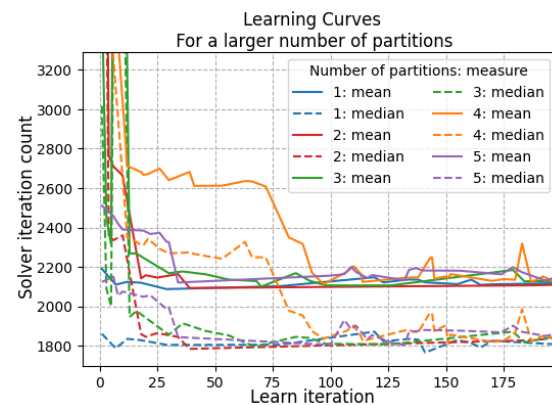


Figure 24: Learning curves for the test with fewer number of partition, the same data as figure 23, but zoomed in to the lower left corner to better see the faster learning rates.

The learning, seen in figures 23 and 24, as expected with fewer partitions has much faster learning and it plateaus almost immediately, with the slowest being the run with 4 partitions, orange learning curve, plateauing at the 100th learning iteration. The problem where accepting a worse state and not being able to get back to a similar solver iteration counts does not happen for these runs, as can be seen just after the 800th learning iteration there is a major spike in solver iteration counts but all tests get back to previous iteration counts at around iteration 900.

In the evaluation of these learned preconditioners, figure 25 and 26, we can see that they are all on par or slightly better than Jacobi preconditioning for all three measure. With using 1 partition begin best in this seeding.

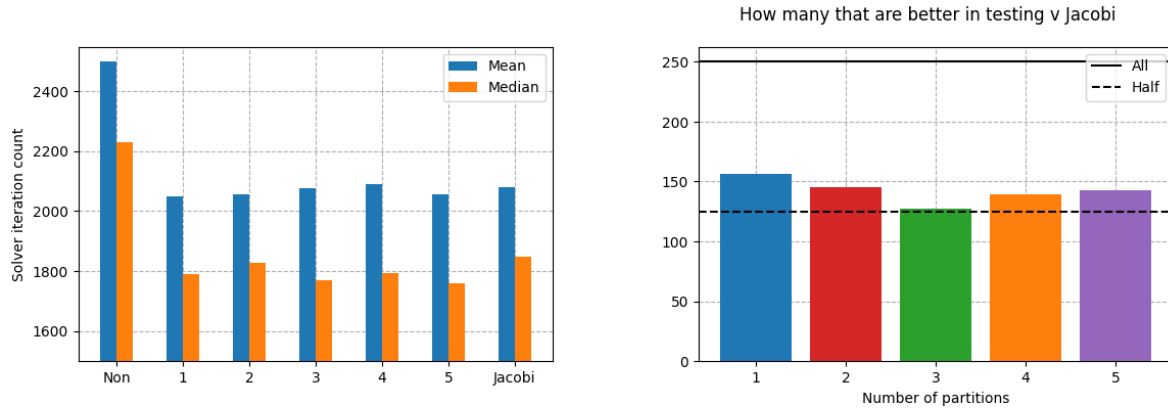


Figure 25: Mean and median solver iteration count from the evaluate of the 5 learned preconditioners, has the same structure as figure 21

Figure 26: Individual linear system comparison between the learned preconditioners and the Jacobi preconditioner, with the same structure as figure 22

The coefficients of the learned preconditioners with fewer number partitions, table 11, are not equal, because of the inheritant randomness of the algorithm, but the values are close with an average value of -0.013641246 and standard deviation of 0.00445245 . Because of this and the increased speed in learning. The new standard preconditioner will have 1 partition in the super diagonal.

Par	Partition coefficients
1	$[-0.01463557]$
2	$[-0.01678746, -0.02122048]$
3	$[-0.00676193, -0.01237617, -0.01208201]$
4	$[-0.02033992, -0.01883074, -0.01200618, -0.01339265]$
5	$[-0.01600863, -0.00575484, -0.00860635, -0.01501886, -0.0107969]$

Table 11: Table of the learned preconditioners coefficients, with a row for each different number of partitions.

From both of the test with changing the number of partitions we did not solve the problem of the preconditioner plateauing, as was the intention with the tests, but we did confirm that using more partitions would result in slower learning to the point of a good preconditioner being unlearnable and with fewer plateauing very quickly.

3.8 Number of diagonals

The next change we will test is the is the number of super diagonals with nonzero value. The initial reason for using only the first super diagonal was to keep the preconditioner as spares as possible, to decreases the computational cost of the matrix vector product for the preconditioner. Because for a matrix in csr format cost depend of the number of nonzero values in the matrix [1]. For these runs we will test with $\{1, 2, 3, 4, 5\}$ super diagonals with one partition in each, as to keep the number of coefficients low for faster learning.

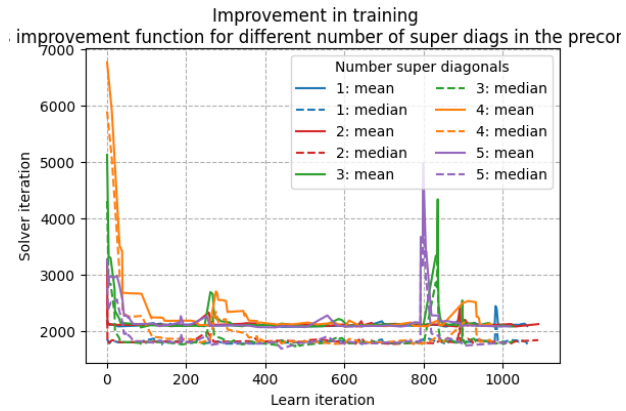


Figure 27: Learning curves for different number of super diagonals in the preconditioner, with line style indicating mean or median solver iteration counts.

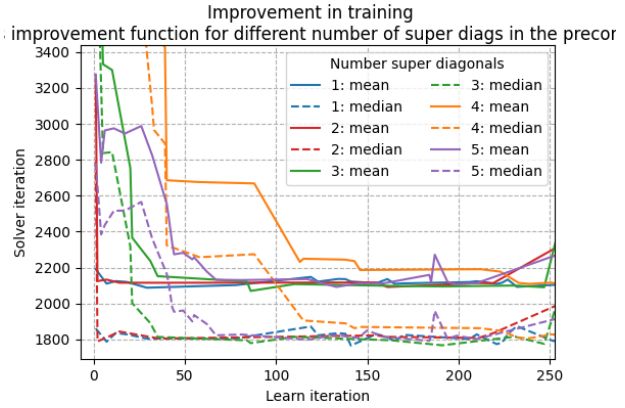


Figure 28: Learning curves for different number of super diagonals in the preconditioner, same data as figure 27 but zoomed in to the lower left corner to see initial learning.

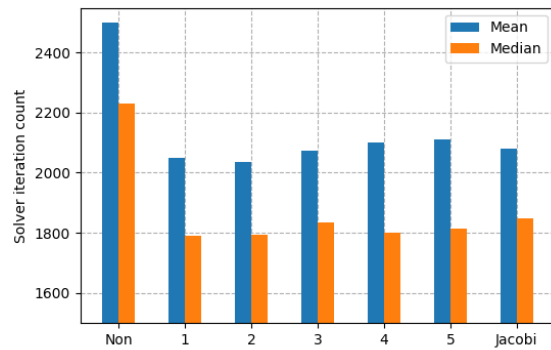


Figure 29: Mean and median solver iteration count for the evaluation of the learned preconditioners with different number of super diagonals, with unpreconditioned and Jacobi preconditioned solver iteration count added for Comparison

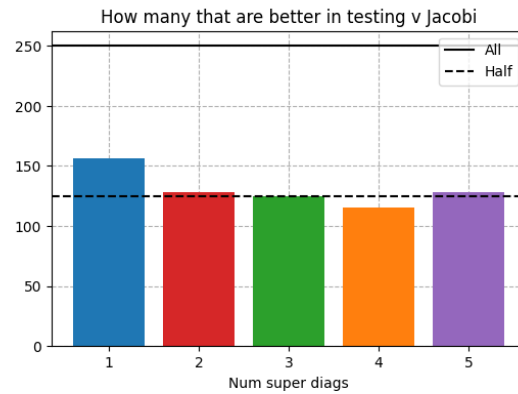


Figure 30: Bar graph showing how many individual different number of super diagonals, with unpreconditioned systems have a lower solver iteration count compared to Jacobi preconditioning.

From the learning curves, figure 27 and 28, we see similar results as the previous test, with regards to the speed of the learning and marginal differences in mean and median solver iteration counts. This can also be seen in the evaluation of the preconditioners with higher number of super diagonal being slightly worse.

In the evaluation of the learned preconditioners it is as expected with what was seen in the learning curves. Marginal differences in the mean and median solver iteration counts, figure 29, and in the individual iteration counts comparison, figure 30, having 1 super diagonal is slightly better. Meaning we conclude that using 1 super diagonal, with no significant difference in evaluation, the faster learning, and most importantly the faster matrix vector product by having fewer nonzero value.

3.9 Super- and subdiagonals

As using more super diagonals did not improve the preconditioner, we will go in the other direction by using both the super and sub diagonals. We will test using (1,2,3,4) super and sub diagonals with 1 partition in each diagonal, leading to (2,4,6,8) different coefficients that needs to be learned. Immediately from the

learning curves in figure 31 seeding 0, we see a sizeable improvement. With the plateaued mean solver iteration count around 1750 and median around 1500, compared with the runs of previous section 3.8 having plateaued around mean of 2100 and median of 1800 solver iteration count. Because of the large improvement, the test was ran again with a different seeding to ensure improvement was not inherent to the seed of the data set and start conditions of the preconditioners. In the learning curves of this second run we see that the learning curves plateaus at similar solver iteration counts, with the only anomaly being the run with 4 super and 4 sub diagonals, the grey curve. This most likely happened because in implementation the coefficients of the preconditioner is first generated, it then tries to change one coefficients to see if an improvement occurred, but in the first learning iteration there is no point of comparison, meaning every state is accepted if all preconditioned systems converged and there is the problem. With bad starting coefficients there may not be a change that can be made to the coefficients such that all systems to converge. Even with the bad start and the slower learning, because of the 8 coefficients that needs to be learned, its learning curve still gets almost as good results as the runs in this test and a better results compared test from the previous sections.

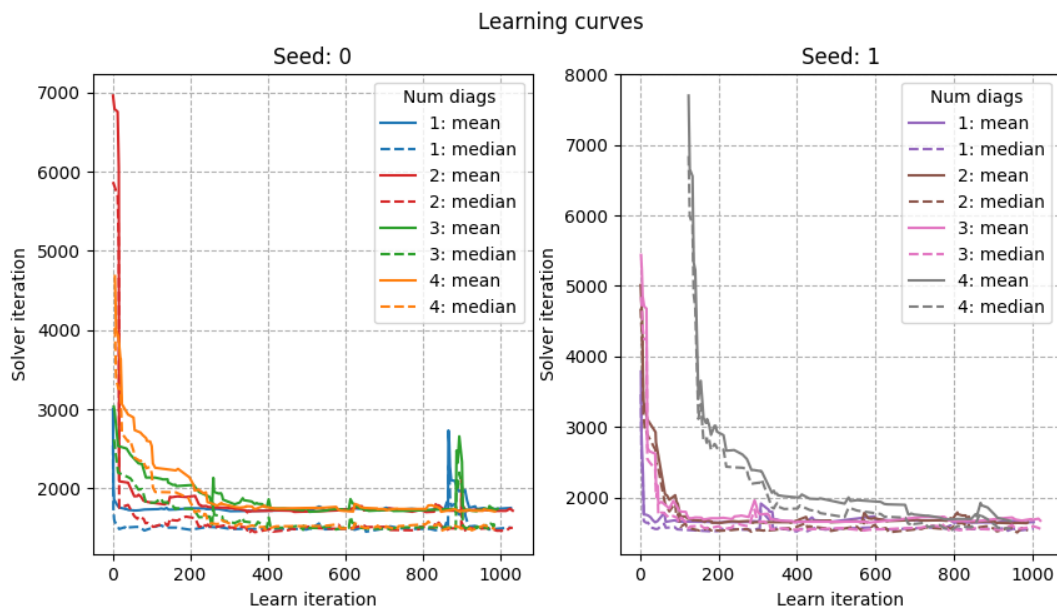


Figure 31: Learning curves for the preconditioner that uses both the sub and super diagonal with a plot for each of the two seedings

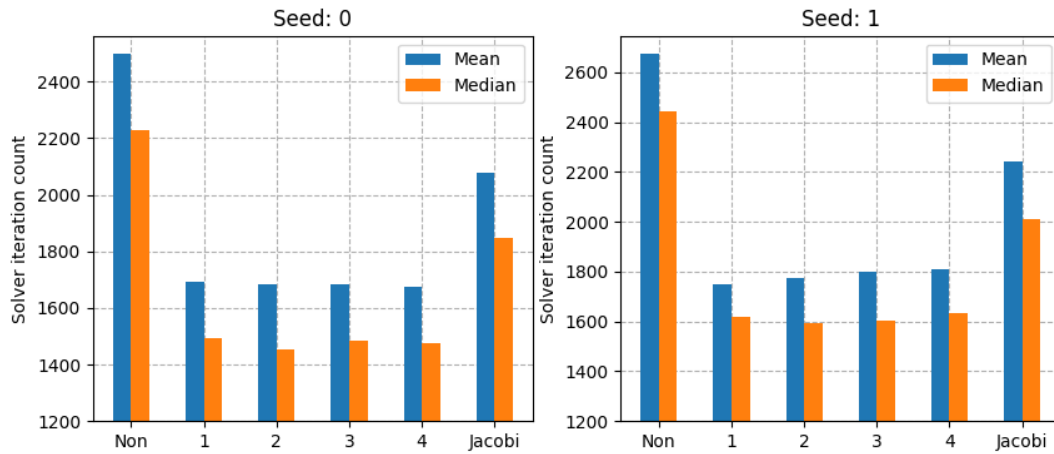


Figure 32: Mean and median solver iteration count for the evaluation of the learned preconditioners with different number of super and sub diagonals, with a plot for each of the two seedings.

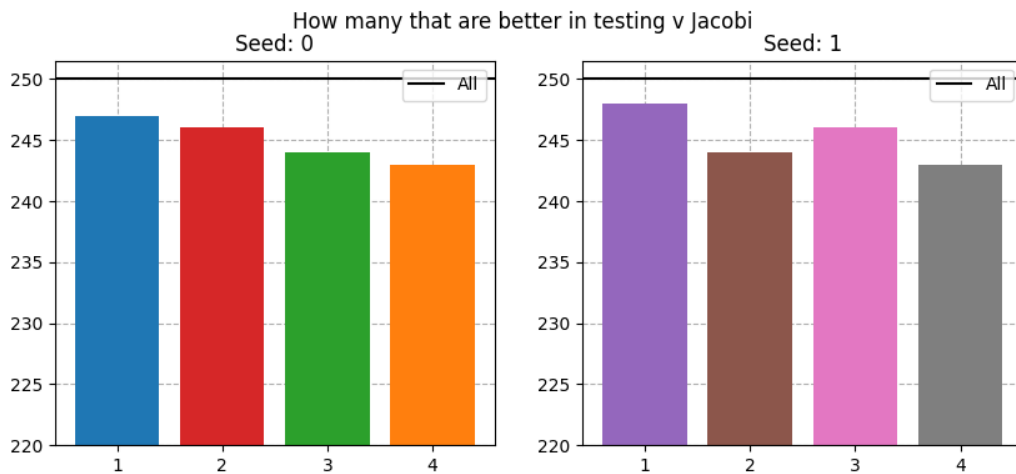


Figure 33: Graph showing how many individual systems that has a lower solver iteration count with the learned preconditioner compared with its Jacobi preconditioned counterpart, with a plot for each of the two seedings.

In the evaluation of the learned preconditioners there also is a major improvement. The mean and median solve iteration counts are around 400 iterations fewer compared to Jacobi preconditioning for all diagonal counts and both seeds, whereas the previous preconditioners with only super diagonal was on par with Jacobi preconditioning, figure 32. And in the individual comparison, figure 33, there is a similar improvement as all preconditioners have more than 240 systems with lower iteration count compared to Jacobi preconditioning, with 1 super and 1 sub diagonal being best for both seedings.

3.9.1 Number of partitions

Because of the significant change made to the preconditioner, we will test if the number of partitions now will have any effect on the learned preconditioners. The number of partitions will be symmetric for the super and sub diagonal and the number of partitions in the test will be $\{1, 2, 3, 4\}$.

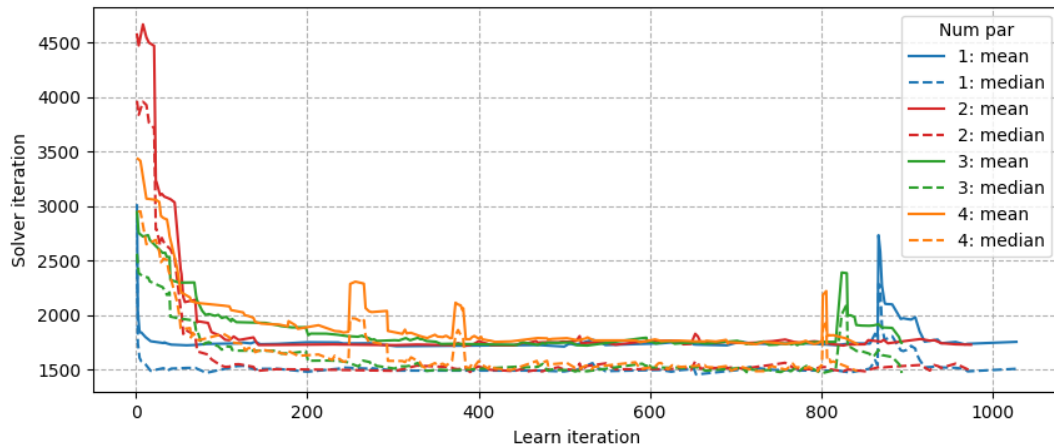


Figure 34: Learning curves for learning preconditioners with 1 super and sub diagonal and different number of partitions.

From the learning curves, figure 34, we see similar results as the previous test with different number of partitions in section 3.7. That is more partitions decrease the learning rate with marginal differences in outcome, with that one partition count is better in one measure but worse in another. For example with 1 partition has the highest number of individual systems that has a lower solver iteration count compared to Jacobi preconditioned systems, figure 36, but has a worse mean and median solver iteration count compared to the learned preconditioner with 2 partitions.

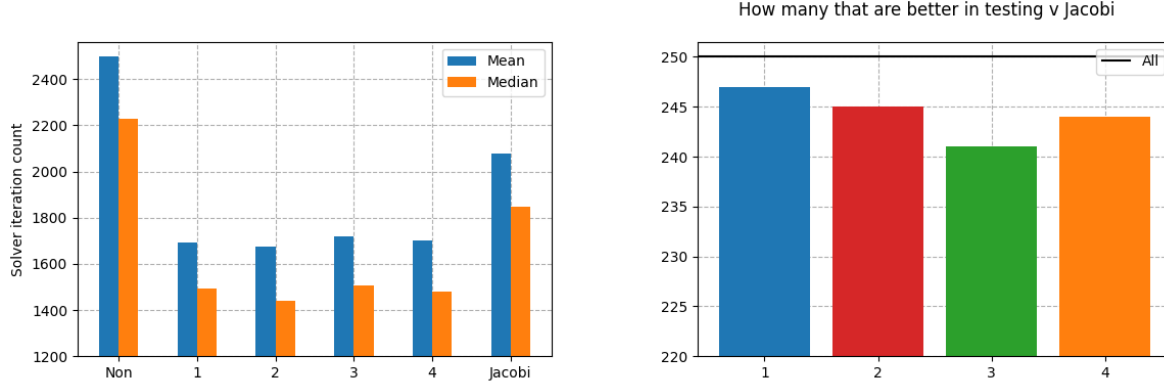


Figure 35: Mean and median solver iteration counts **Figure 36:** Graph showing how many individual systems from the evaluation of the learned preconditioners, tems that has a lower solver iteration count with the with unpreconditioned and Jacobi preconditioning for learned preconditioners compared with its Jacobi preconditioned counterpart.

Par	Super diagonal	Sub diagonal
1	$[-0.059248]$	$[-0.054830]$
2	$[-0.063006, -0.064121]$	$[-0.059046, -0.056173]$
3	$[-0.064320, -0.065399, -0.058531]$	$[-0.055929, -0.074727, -0.055672]$
4	$[-0.046214, -0.058072, -0.047579, -0.063756]$	$[-0.044746, -0.060495, -0.049910, -0.067014]$

Table 12: Table of the learned preconditioners coefficients, with a row for each different number of partitions, columns for the super diagonal coefficients and one for the sub diagonal coefficients.

For the coefficients of the learned, we again see similarities to section 3.7, that being the coefficients are similar, even between the the super and sub diagonal, table 12. Because of this, with only slight differences in learning outcome and the faster learning we can again conclude that using 1 partition each diagonal gets better learned preconditioners.

3.9.2 Symmetry

Because of the similarities of the coefficients between the super and sup diagonal, we will repeat the test from section 3.9 with symmetric preconditioners, as this would half the number of coefficients compared to the test from that sections preconditioners.

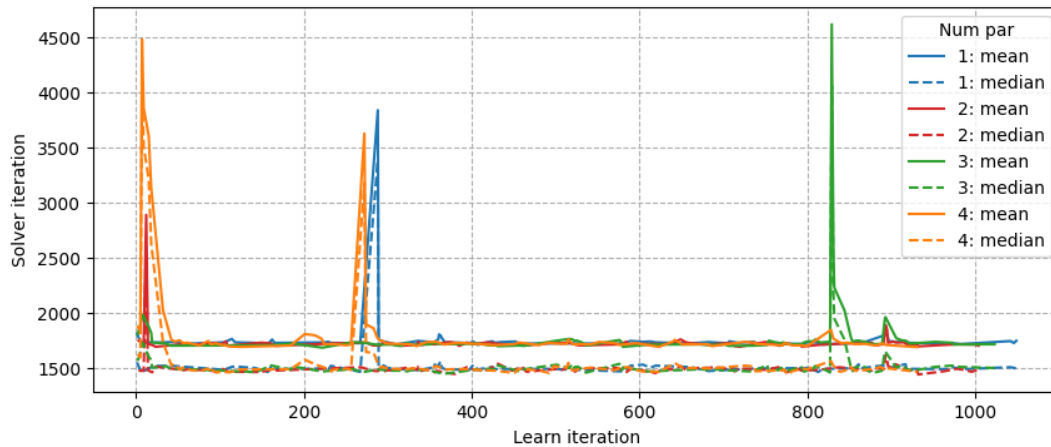


Figure 37: Learning curves for from the Learning of symmetric preconditioners with 1 super and sub diagonal.

On major difference in the learning for all of these runs is the run time of each of the configurations, as the previous two test had a run time of 3 hours for one learning configuration and for all four of these had was about 5 hours. The cause of the increase in run time is that changes made to the preconditioners during learning increased the solver iteration counts significantly of most of the systems, as can be seen as the spikes in figure 37 where such a state was accept. The cause of large increase in solver iteration count because the change made to a coefficient has a larger affect on the preconditioners. A solution would entail a change in the hyperparameter regrading the distribution generating the change made to the coefficients, but changes to this distribution would have effects on the learning time.

In the evaluation of the learned preconditioners, figure 38 and 39, there is only marginal differences between the solver iteration counts of these tests and the test with unsymmetric Preconditioners in section 3.9 seeding 0. With this we can conclude in this parameter configuration not using a symmetric preconditioner is the better option.

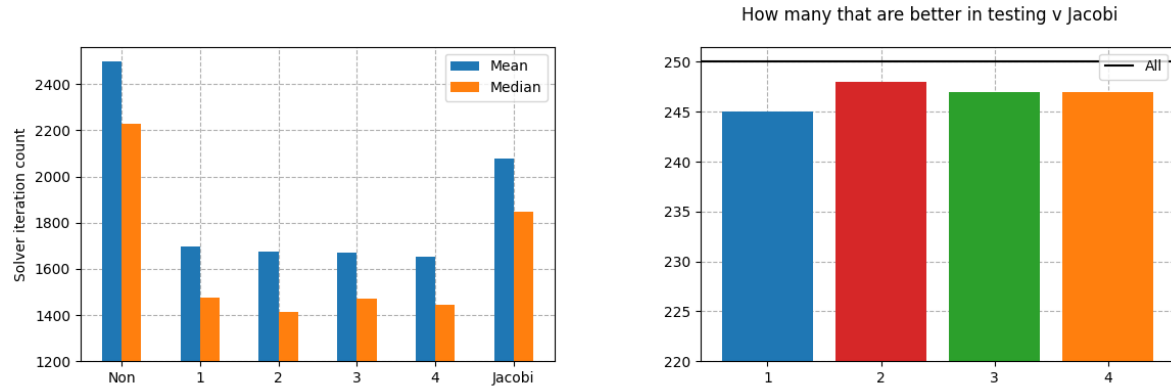


Figure 38: Mean and median solver iteration counts **Figure 39:** Bar graph showing how many individual from the evaluation of the learned preconditioners, systems that has a lower solver iteration count with with unpreconditioned and Jacobi preconditioning for the learned preconditioners compared with its Jacobi preconditioned counterpart.

3.10 Testing the algorithm on larger systems

With the learning algorithm and structure of the preconditioner gotten to a point where it can reliably learn a preconditioner than can improve the mean solver iteration count by 2000 iterations compared to a trivial preconditioner. There is one parameter that have not changed and that is the size of the 2-dimensional laplace operator used in the training and test data sets. Because no testing has been done on different size laplacian operators, importantly larger sizes. The conclusions made from the different tests, especially on the parameter choice and the structure of preconditioner, might be specialized towards the current system size and perturbations. One change that as made to the structure of the preconditioner that is specialized, was made in section 3.5, and that is the change from an identity diagonal to the Jacobi preconditioner as the diagonal, because if the matrices in our data sets had diagonals closer to an identity diagonal Jacobi preconditioning would make not any difference or if the diagonals were close to zero Jacobi preconditioner would have and adverse effect on the preconditioner, and it could therefore struggle to learn a good preconditioner.

Therefore the next test we will change the size of the systems sizes and the perturbations applied to them. Unfortunately this is not as simple as it would seem, because of the condition in our learning algorithm that all systems must converge for a change made to the preconditioner to be accepted. This condition was original implemented because of the problem of comparison between learning iterations where different system does not converge, as it could skew with results of the comparisons in either direction.

To find a suitable larger size and larger perturbations, preliminary tests testing different size and perturbation distribution data sets on the linear solvers. In these preliminary tests there were almost always at least one or two systems that did not converge, we concluded that it is because of the perturbation distribution being the normal distribution having larger variance and larger system sizes there is more perturbations that is needed to be generated and therefore more likely generate an outlier. To get more control of outliers we changed the distribution that generate the perturbations to a uniform distribution with mean of 1 and variable range, $Uni(1 - r, 1 + r)$. Testing with this distribution we landed on the 1-dimensional discrete laplacian operator size of 65 resulting in a final system size of 4225 and a range of 0.45 for the perturbations. With these data parameters we ran the learning algorithm with 50 base learning iterations on 4 different seedings and preconditioner having 1 super and 1 sub diagonal with 1 partition each.

N	Data Dist	Amount	Learn iter	Learn Scales	Precond dist	Par	Diagonals
65	$Uni(0.55, 1.45)$	250	50	$\pm\{1, 0.5, 0.25\}$	$N(0, 0.05^2)$	1	$(1, -1)$

Table 13: Table of the run parameter configuration where the column corresponds to in order the size of the 1-dimension discrete laplace operator; the initial perturbations on the data; the amount of systems in the data sets; the base number of learning iterations; the scales of the exploration changes; the distribution of the initial coefficients; the number of partitions; the diagonal used.

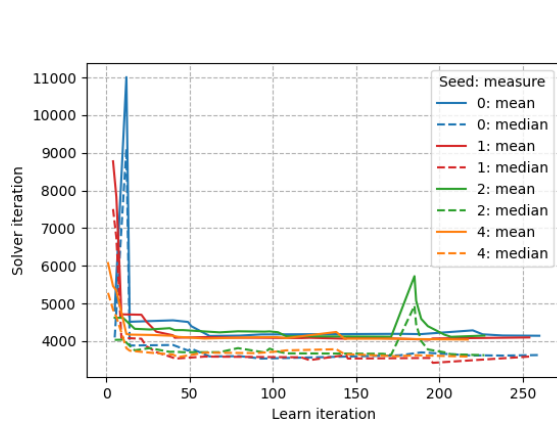


Figure 40: Learning curves for from the learning of the larger systems for the different seedings

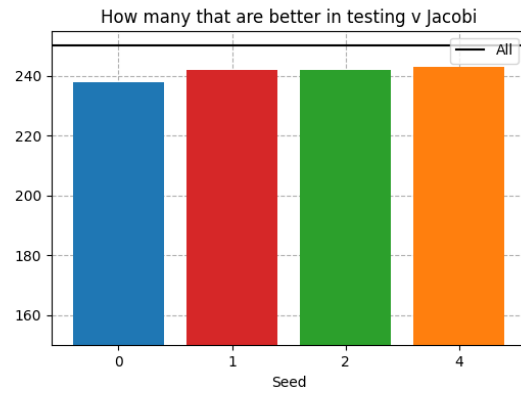


Figure 41: Graph showing how many individual systems that has a lower solver iteration count with the learned preconditioners compared with its Jacobi preconditioned counterpart from the different seedings.

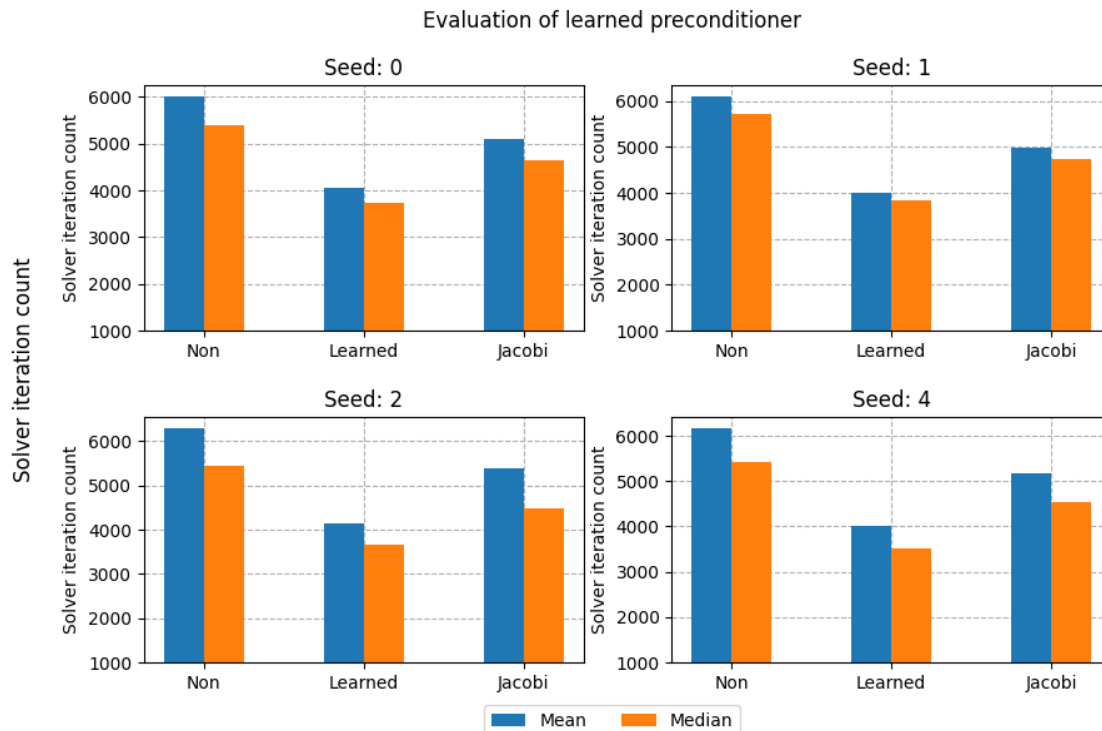


Figure 42: Mean and median solver iteration counts from the evaluation of the learned preconditioners, with unpreconditioned and Jacobi preconditioning for comparison, with a plot for each of the seedings.

From learning curves in figure 40 we can see what we have seen many times in the previous tests, that is the mean and median solve iteration counts decrease during the learning. For example for seeding 1, red curve, starts with a mean solver iteration count 8773.2 and decreases to a count a bit above 4000, and similar curves for the other seedings. From this we can say that the learning the coefficients regardless of the four sets starting coefficients.

In the evaluation of the learned preconditioners, we can see that the preconditioners decreases the mean and median solver iteration count by approximately 2000 iterations compared to using no preconditioner and 1000 compared with Jacobi preconditioning, figure 42. For the individual comparisons in figure 41 that around 240 system the learned preconditioner is better than using Jacobi preconditioning. Meaning it is fair to say that the algorithm has learned a good preconditioner for the larger system sizes for all four seeding.

4 Conclusion

5 Appendix

6 Bibliography

- [1] Zhaojun Bai et al. *Templates for the Solution of Algebraic Eigenvalue Problems*. Ed. by Zhaojun Bai et al. Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719581. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9780898719581>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898719581>.
- [2] Richard Barrett et al. "Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods". In: *Mathematics of Computation* 64 (Sept. 1996). DOI: 10.2307/2153507.

- [3] Alexandre Boutot and Viraj Dsouza. *Explicit Block Encodings of Discrete Laplacians with Mixed Boundary Conditions*. 2026. arXiv: 2603.12405 [quant-ph]. URL: <https://arxiv.org/abs/2603.12405>.
- [4] Y. D. Chong. *Computational Physics*. LibreText. URL: [https://phys.libretexts.org/Bookshelves/Mathematical_Physics_and_Pedagogy/Computational_Physics_\(Chong\)](https://phys.libretexts.org/Bookshelves/Mathematical_Physics_and_Pedagogy/Computational_Physics_(Chong)).
- [5] Nicholas J. Higham. *The Power of Bidiagonal Matrices*. 2023. arXiv: 2311.06609 [math.NA]. URL: <https://arxiv.org/abs/2311.06609>.
- [6] *Matrix market*. URL: <https://math.nist.gov/MatrixMarket/>.
- [7] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Second. Society for Industrial and Applied Mathematics, 2003. DOI: 10.1137/1.9780898718003. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9780898718003>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898718003>.
- [8] Henk Van der Vorst. “Iterative Krylov Methods for Large Linear Systems”. In: 13 (Apr. 2003). DOI: 10.1017/CB09780511615115.